## Contents


0.1 图论 uptate: 2026.4.28 . . . 2
0.2 一些概念 . . . 2
0.3 树的重心 . . . 3
0.4 树的直径 . . . 4
0.5 重链剖分 . . . 5
0.6 树链剖分（线段树） . . . 7
0.7 FastLCA（欧拉序 +RMQ） . . . 9
0.8 倍增求 LCA . . . 11
0.9 树上差分 . . . 12
0.10 点分治 . . . 12
0.11 树上启发式合并 . . . 13
0.12 基环树与基环树森林 . . . 14
0.13 虚树 . . . 17
0.14 拓扑排序 . . . 18
0.15 树哈希 . . . 19
0.16 最短路 . . . 21
0.16.1 单源最短路 . . . 21
0.16.2 全源最短路 . . . 30
0.16.3 多源最短路 . . . 36
0.16.4 最长路 . . . 36
0.17 生成树 . . . 37
0.17.1 最小生成树 . . . 37
0.17.2 kruskal 重构树 . . . 39
0.17.3 . . . 41
0.18 网络流 . . . 42
0.18.1 拆点 . . . 42
0.18.2 最大流 Dinic . . . 42
0.18.3 最小费用最大流 . . . 44
0.19 连通性 . . . 46
0.19.1 有向图连通性 . . . 46
0.19.2 无向图连通性 . . . 50
0.20 欧拉图 . . . 56
0.20.1 无向图 . . . 56
0.20.2 有向图 . . . 58
0.20.3 混合图 . . . 58
0.21 拉姆齐定理 . . . 60
0.22 竞赛图 . . . 60
0.23 二分图 . . . 61
0.23.1 二分图的判断 . . . 61
0.23.2 二分图上博弈 . . . 62
0.23.3 删边二分图 . . . 62
0.23.4 情况一：K = 0 . . . 62
0.23.5 情况二：K > 0 . . . 62
0.24 图匹配 . . . 64
0.24.1 基础概念 . . . 64
0.24.2 二分图上的匹配 . . . 65
0.24.3 一般图最大匹配 . . . 75
0.25 环计数问题 . . . 78
0.25.1 无向图简单环计数 . . . 78
0.25.2 无向图三元环计数 . . . 78
0.25.3 无向图四元环计数 . . . 80
0.26 最大团与最大独立集 . . . 82

## 0.1 图论 uptate: 2026.4.28

## 0.2 一些概念

基图：即把有向图中的有向边改为无向边得到的图称作基图

弱连通：对于有向图中两点 $(u, v)$，如果我们转化为基图后 $(u, v)$ 联通, 那么称 $(u, v)$ 弱联通

完全图: 任意两点间都有一条无向边的图称作完全图，$n$ 个点的完全图记作 $K_n$

子图：对于一个图 $G = (V, E)$，如果另一个图 $G' = (V', E')$ 满足 $V' \quad V$ 且 $E' \quad E$ 则 $G'$ 称作 $G$ 的子图

导出子图: 选定一个点集 $V$，再加上两端都在 $V$ 中的边，即为点集 $V$ 的导出子图

平面图: 将所有点和边铺在平面上，可以满足每条边之间两两不相交的图称作平面图，对于平面图 $G = (V, E)$ 满足 $E \leq 3V - 6$ 且根据欧拉公式，平面图满足 $V - E + F = C + 1$ 其中 $F$ 是面的个数,$C$ 是图连通块的个数

### 0.3 树的重心

我们不妨设树的大小为 $n$

树的重心定义为，当树以重心为根时，每个子树的大小不超出 $n/2$。

即最大的子树大小 $siz * 2 <= n$

性质：一棵树最多只能有两个重心。

求重心的方法：

在我们求出以任意一点为根的子树大小后，我们从根开始 $dfs$。如果某个点的所有儿子的子树大小都 $\leq n/2$，那么该点为重心，直接返回。否则我们递归子树大小超过 $n/2$ 的儿子即可（显然只有一个）。

```
struct Cent{
    int n;
    vector<vector<pair<int, int>>> g; // 邻接表
    vector<int> siz;                  // 子树大小
    Cent(int n) : n(n), g(n + 1), siz(n + 1) {}
    void addEdge(int u, int v, int w){
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }
    void dfsSize(int u, int fa){
        siz[u] = 1;
        for (const auto &[v, w] : g[u]){
            if (v != fa){
                dfsSize(v, u);
                siz[u] += siz[v];
            }
        }
    }

    int findCentroid(int u){
        dfsSize(u, 0);
        return getRoot(u, 0, siz[u]);
    }
private:
    int getRoot(int u, int fa, int total_size){
        for (const auto &[v, w] : g[u]){
            if(v == fa) continue;
            if (2 * siz[v] > total_size){
                return getRoot(v, u, total_size);
            }
            if (2 * siz[v] == total_size){// 返回编号最小的重心
                return min(u,v);
            }
        }
        return u;
    }
};
```

### 0.4 树的直径

注意初始化时传入的图因该是 1-base
可直接获取直径长度，其中一条直径上的所有点，以及每个点到父节点直接连边的权值
两次 bfs 会更改 fa[] 数组，其中 x 表示返回路径的起点，y 表示终点，d 是直径长度

```
template <class T>
struct Diam{
    int n;
    vector<vector<pair<int, int>>> g;           // 邻接表（无向图）
    vector<int> fa;                             // 父节点记录（用于路径回溯）
    Diam(int n) : n(n), g(n + 1), fa(n + 1) {}
    void addEdge(int u, int v, int w){
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }
    // BFS 求最远节点（返回节点编号）
    pair<int, T> bfs(int start){
        vector<T> dep(n + 1, -1);
        queue<int> q;
        q.push(start);
        dep[start] = 0;
        fa[start] = -1;
        while (!q.empty()){
            int u = q.front();
            q.pop();
            for (auto [v, w] : g[u]){
                if (dep[v] == -1){
                    dep[v] = dep[u] + w;
                    fa[v] = u;
                    q.push(v);
                }
            }
        }
        int maxPos = max_element(dep.begin(), dep.end()) - dep.begin();
        return make_pair(maxPos, dep[maxPos]);
    }
    // 求树的直径（返回直径长度和端点 {u, v}）
    pair<T, pair<int, int>> getDiam(){
        auto [x, dis1] = bfs(1); // 第一次BFS
        auto [y, dis2] = bfs(x); // 第二次BFS
        return {dis2, {x, y}};
    }
    // 获取树直径上的所有点first 以及 每个点到父节点路径的权值second
    vector<int> getPathDia(){
        auto [dis,uv] = getDiam();
        auto [u,v] = uv;
        vector<int> path(1,v);
        while(u!=v){
            v = fa[v];
            path.push_back(v);
        }
        return path;
    }
};
```

直径的性质:

- 树上任意一点，离它最远的一点必然是直径两端点之一
- 两个联通块合并，新联通块的直径两端点一定是原先的四个直径端点中的其中两个
- 我们可以扩展 2 中的结论，我们将直径的定义扩展为点权 + 边权的情况下仍然成立
- 一棵树可以拥有多条直径，其所有直径共用一个中点，中点有可能在边上 (两个相邻端点的边上)

在树上选择一条边长度和不超过 s 的路径使其他所有结点到这条路径的距离的最大值最小，那么选择的这条路径在树的直径上，可以用滑动窗口 +st 表做, 窗口的两个端点处只需要维护直径上前后缀的长度即可

### 0.5 重链剖分

重链剖分可以将树上的任意一条路径划分成不超过 $O(logn)$ 条连续的链，每条链上的点深度互不相同。同时会给树的点重新编号为 $dfn$ 序。$dfn$ 序的定义: 按先遍历重儿子的顺序得到的 $dfs$ 序。

**重链剖分的性质:**

1：每条重链上的 $dfn$ 序连续。

2：每个点子树内的 $dfn$ 序连续，节点 $u$ 子树内的 $dfn$ 序范围是 $[dfn[u], dfn[u] + siz[u] - 1]$

3：每个点到根节点的路径上，最多只会有 $O(logn)$ 个轻儿子。

4：每条路径能被划分为最多 $O(logn)$ 条重链。

关于第三个和第四个性质的证明:

每次经过轻儿子，子树大小至少会减半，因此最多下降 $O(logn)$ 次。

**使用时自己新开数组的话一定记得在构造函数那里分配内存!!!**

```
template <class T>
struct TreePre{
    int n,idx = 0,root;
    vector<vector<pair<int, int>>> &g;
    void dfs1(int u, int f){
        fa[u] = f;
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if(v == f) continue;
            dep[v] = dep[u] + 1;
            dis[v] = dis[u] + w;
            dfs1(v, u);
            siz[u] += siz[v];
            if (siz[son[u]] < siz[v]){
                son[u] = v;
            }
        }
    }
    void dfs2(int u, int tp){
        dfn[u] = ++idx;
        idfn[idx] = u;
        top[u] = tp;

        if (son[u]) dfs2(son[u], tp);

        for (auto [v, w] : g[u]){
            if (v != fa[u] and v != son[u]){
                dfs2(v, v);
            }
        }
    }

public:
    TreePre(vector<vector<pair<int, int>>> &g, int root)
        : g(g), n(g.size() - 1), root(root), dep(n + 1),
        top(n + 1), son(n + 1), fa(n + 1), siz(n + 1),
        dfn(n + 1), idfn(n + 1), dis(n + 1)
    {
        dep[root] = 1;
        dis[root] = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }
    vector<int> dfn, idfn, siz, fa, dep, top, son;
    vector<T> dis;
    int kthAncester(int u, int k) // k级祖先
    {
        if (dep[u] <= k)
        {
            return -1;
        }
        int d = dep[u] - k;
        while (dep[top[u]] > d)
        {
            u = fa[top[u]];
        }
        return idfn[dfn[u] - dep[u] + d];
    }
    int getLca(int u, int v) // u和v在指定根下的lca
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] > dep[top[v]])
            {
```

```
                u = fa[top[u]];
            }
            else
            {
                v = fa[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    T getDistance(int u, int v) // u和v之间的距离
    {
        return dis[u] + dis[v] - 2 * dis[getLca(u, v)];
    }
    bool isAncester(int u, int v) // u是不是v的祖先
    {
        return dfn[u] <= dfn[v] and dfn[u] + siz[u] - 1 >= dfn[v];
    }

    auto getRoad(int u, int v) // 得到uv之间路径的dfn序
    {
        vector<pair<int, int>> res;
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
            {
                swap(u, v);
            }
            res.push_back({dfn[top[u]], dfn[u]});
            u = fa[top[u]];
        }
        if (dep[u] > dep[v])
        {
            swap(u, v);
        }
        res.push_back({dfn[u], dfn[v]});
        return res;
    }

    int rootedFather(int u, int root) // u在root为根的情况下的父节点
    {
        if (!isAncester(u, root)) // u不是root的祖先 那么父节点不变
        {
            return fa[u];
        }
        auto it = upper_bound(g[u].begin(), g[u].end(), root, [&](int x, int y)
                              { return dfn[x] < dfn[y]; });
        --it;
        return *it;
        // 把u的邻点按dfn序排序后 dfn最大且  dfn[u]<=dfn[root]的点
    }

    int rootedSize(int u, int root) // root为根的情况下u的子树大小
    {
        if (root == u)
        {
            return n;
        }
        if (!isAncester(u, root)) // u不是根的祖先
        {
            return siz[u];
        }
        return n - siz[rootedFather(u, root)];
    }
    int rootedLca(int root, int u, int v)
    {
        return getLca(u, v) ^ getLca(u, root) ^ getLca(v, root);
    }
};
```

### 0.6 树链剖分（线段树）

配合区间修改线段树修改树上点权，如果修改边权可以让每个点的点权表示这个点指向父节点的边的权，那么让 x 到 y 路径的边权区间加可以先 addRoad(x,y,v), 再 addRoad(lca,lca,-v);

```
template <class T>
class TreePre
{
public:
    int n;
    int idx = 0;
    int root;
    std::vector<std::vector<std::pair<int, int>>> &g;
    std::vector<int> dfn, idfn, siz, fa, dep, top, son;
    std::vector<T> dis;
    void dfs1(int u, int f)
    {
        fa[u] = f;
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if (v != f)
            {
                dep[v] = dep[u] + 1;
                dis[v] = dis[u] + w;
                dfs1(v, u);
                siz[u] += siz[v];
                if (siz[son[u]] < siz[v])
                {
                    son[u] = v;
                }
            }
        }
    }
    void dfs2(int u, int tp)
    {
        dfn[u] = ++idx;
        idfn[idx] = u;
        top[u] = tp;
        if (son[u]){
            dfs2(son[u], tp);
        }
        for (auto [v, w] : g[u]){
            if (v != fa[u] and v != son[u])
            {
                dfs2(v, v);
            }
        }
    }
    TreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root)
        : g(g), n(g.size() - 1), root(root), dep(n + 1), top(n + 1), son(n + 1),
            fa(n + 1), siz(n + 1), dfn(n + 1), idfn(n + 1), dis(n + 1)
    {
        dep[root] = 1;
        dis[root] = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }

    int getLca(int u, int v) // u和v在指定根下的lca
    {
        while (top[u] != top[v]){
            if (dep[top[u]] > dep[top[v]]){
                u = fa[top[u]];
            }
            else{
                v = fa[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    // 对树上的路径操作
    void addRoad(int x,int y,int v,SegmentTree<T>& tr){
        while(top[x] != top[y]){
            if (dep[top[x]] > dep[top[y]]){
                tr.rangeAdd( dfn[top[x]],dfn[x], v);
                x = fa[top[x]];
            }
            else{
                tr.rangeAdd( dfn[top[y]], dfn[y],v);
                y = fa[top[y]];
            }
```

```
        }
        if(dep[x] > dep[y]) swap(x,y);
        tr.rangeAdd(dfn[x], dfn[y], v);
    }
    T queryRoad(int x,int y,SegmentTree<T>& tr){
        int res = 0;
        while(top[x] != top[y]){
            if (dep[top[x]] > dep[top[y]]){
                res += tr.rangeQuerySum( dfn[top[x]],dfn[x]);
                res %= mod;
                x = fa[top[x]];
            }
            else{
                res += tr.rangeQuerySum( dfn[top[y]], dfn[y]);
                res %= mod;
                y = fa[top[y]];
            }
        }
        if(dep[x] > dep[y]) swap(x,y);
        res += tr.rangeQuerySum(dfn[x], dfn[y]);
        res %= mod;
        return res;
    }
    // [dfn[x], dfn[x]+siz[x]-1]为x子树的所有节点
    void addSubtree(int x,int z,SegmentTree<T>& tr){
        tr.rangeAdd(dfn[x], dfn[x]+siz[x]-1, z);
    }
    T querySubtree(int x,SegmentTree<T>& tr){
        int res = tr.rangeQuerySum(dfn[x], dfn[x]+siz[x]-1);
        return res;
    }
};
```

### 0.7 FastLCA（欧拉序 +RMQ)

我们考虑用 $dfs$ 序求两个点的 $LCA$

结论：不妨设 $dfn[x] < dfn[y]$，那么 $lca(x,y)$ 是 $dfn$ 序上 $[dfn[x]+1, dfn[y]]$ 区间内深度最小的点的父亲

正确性显然：按 $x$ 和 $y$ 是否有祖先关系讨论即可。

用返回最小位置的 $ST$ 表预处理可以做到 $O(1)$ 查询。配合 ST 表板子食用 （ST 板子记得改成返回权值最小的下标）

重链剖分的预处理 TreePre 只需要 dfs1,dfs2 和构造函数就够了

```
template <class T>
struct TreePre{
    int n,idx = 0,root;
    vector<vector<pair<int, int>>> &g;
    void dfs1(int u, int f){
        fa[u] = f;
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if(v == f) continue;
            dep[v] = dep[u] + 1;
            dis[v] = dis[u] + w;
            dfs1(v, u);
            siz[u] += siz[v];
            if (siz[son[u]] < siz[v]){
                son[u] = v;
            }
        }
    }
    void dfs2(int u, int tp){
        dfn[u] = ++idx;
        idfn[idx] = u;
        top[u] = tp;

        if (son[u]) dfs2(son[u], tp);

        for (auto [v, w] : g[u]){
            if (v != fa[u] and v != son[u]){
                dfs2(v, v);
            }
        }
    }
public:
    TreePre(vector<vector<pair<int, int>>> &g, int root)
        : g(g), n(g.size() - 1), root(root), dep(n + 1),
        top(n + 1), son(n + 1), fa(n + 1), siz(n + 1),
        dfn(n + 1), idfn(n + 1), dis(n + 1)
    {
        dep[root] = 1;
        dis[root] = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }
    vector<int> dfn, idfn, siz, fa, dep, top, son;
    vector<T> dis;
};
//  -----------------
template <class T>
class ST{
    vector<vector<int>> st;
    vector<T> A;
public:
    ST(const vector<T> &a){
        A = a;
        int n = a.size() - 1, logn = __lg(n);
        st.assign(logn + 1, vector<int>(n + 1));
        for (int i = 1; i <= n; ++i) st[0][i] = i;
        for (int j = 1; j <= logn; ++j){
            for (int i = 1; i + (1 << j) - 1 <= n; ++i){
                st[j][i] = (a[st[j - 1][i]] < a[st[j - 1][i + (1 << (j - 1))]]
                    ? st[j - 1][i] : st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    int operator()(int l, int r){
        int log = __lg(r - l + 1);
        return (A[st[log][l]] < A[st[log][r - (1 << log) + 1]]
                    ? st[log][l] : st[log][r - (1 << log) + 1]);
    }
};
//  -----------------
template <class T>
```

```
class FastLCA
{
    int n;
    vector<vector<pair<int, int>>> g;
    TreePre<T> pre;
    ST<T> *st;
public:
    FastLCA(vector<vector<pair<int, int>>> &g, int root) : n(g.size() - 1), g(g), pre(g, root)
    {
        vector<int> reLabel(n + 1);
        for (int i = 1; i <= n; ++i){
            reLabel[pre.dfn[i]] = pre.dep[i];
        }
        st = new ST(reLabel);
    }
    int getLca(int u, int v){
        if (u == v) return u;
        u = pre.dfn[u], v = pre.dfn[v];
        if (u > v) swap(u, v);
        return pre.fa[pre.idfn[(*st)(u + 1, v)]];
    }
};
```

### 0.8 倍增求 LCA

O(log(n)) 求 lca , 原理是先把 u 和 v 跳到同一个高度，然后再共同上跳知道最近的 lca

```
void faLCA(int n, int root, vector<vector<int>>& adj, vector<vector<int>>& fa, vector<int>& dep) {
    fa = vector<vector<int>>(n + 1, vector<int>(21));
    dep = vector<int>(n + 1);
    auto dfs = [&](auto self, int u, int ffa) -> void {
        fa[u][0] = ffa;
        dep[u] = dep[ffa] + 1;
        for (int v : adj[u]) {
            if (v == ffa) continue;
            self(self, v, u);
        }
    };
    dfs(dfs, root, root);
    for (int k = 1; k <= 20; k++) {
        for (int i = 1; i <= n; i++) { fa[i][k] = fa[fa[i][k - 1]][k - 1]; }
    }
}

void solve() {
    int n, q, root;
    cin >> n >> q >> root;
    vector<vector<int>> adj(n + 1);
    for (int i = 2; i <= n; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].emplace_back(v);
        adj[v].emplace_back(u);
    }
    // static int fa[500010][21]; 洛谷模板题P3379换成这个才不会t....
    vector<vector<int>> fa;
    vector<int> dep;
    faLCA(n, root, adj, fa, dep);
    auto lca = [&](int u, int v) -> int {
        if (dep[u] < dep[v]) swap(u, v);
        for (int k = 20; k >= 0; k--) {
            if (dep[fa[u][k]] < dep[v]) continue;
            u = fa[u][k];
        }
        if (u == v) return u;
        for (int k = 20; k >= 0; k--) {
            if (fa[u][k] != fa[v][k]) { u = fa[u][k], v = fa[v][k]; }
        }
        return fa[u][0];
    };
    while (q--) {
        int u, v; cin >> u >> v;
        cout << lca(u, v) << "\n";
    }
}
```

## 0.9 树上差分

**点权差分**（$ans[x]$ 表示 $x$ 点的点权）

```
ans[x]++;ans[y]++;
ans[LCA.pre.fa[lca]]-=1;
ans[lca]-=1;
```

**边权差分**（$ans[x]$ 表示 x 到其父节点的边权）

```
ans[x]++;ans[y]++;
ans[lca]-=2;
```

## 0.10 点分治

点分治的思想是统计经过根的路径的贡献，对于不经过根的路径，我们分治下去直至每个点都做为根被统计贡献。在这样的情况下，不断选择树的重心分治是最优的选择，可以证明在该条件下访问点的数量为 $nlogn$ 级别的。也就是说，我们通过一种特殊的遍历顺序得到了所有路径的信息。

特别的，如果我们存下来点分治过程中经过的节点，我们称作点分治序，每个点对应的可结合的点的点分治序都是一个连续的区间。

在以每个点为根统计答案时，我们考虑两种不同的方法进行求解。

不妨设当前根为 $u$,

1. 对于 $u$ 的每一颗子树，我们挨个遍历并且和之前遍历到的子树信息进行结合统计答案。在遍历完一颗子树后要加入该子树的信息。在遍历完所有子树后，要清空信息。这样的方法能保证在统计时就不重不漏。请注意清空信息！

2. 我们直接把所有 $u$ 的信息统计出来并计算答案，注意到这时候可能有单个子树之中的点结合，这部分是非法答案，我们考虑容斥减掉对应的贡献。具体的说，考虑一个函数 $calc(int\ root, int\ val)$ 表示以 $root$ 为根，当前权值为 $val$ 的贡献，我们要加上 $calc(u, 0)$，减去所有的 $calc(v, w(u, v))$

```
struct PointerDivide
{
    vector<vector<pair<int, int>>> &g;
    vector<int> siz, dfn, vis;
    vector<vector<int>> pdt;
    int root;
    void dfsSize(int u, int fa){
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if (v != fa and !vis[v])
            {
                dfsSize(v, u);
                siz[u] += siz[v];
            }
        }
    }
    int getRoot(int u, int fa, int Size){
        for (auto [v, w] : g[u]){
            if (v != fa and !vis[v] and 2 * siz[v] > Size){
                return getRoot(v, u, Size);
            }
        }
        return u;
    }
    void build(int u){
        vis[u] = true;
        dfn.push_back(u);
        for (auto [v, w] : g[u]){
            if (!vis[v]){
                dfsSize(v, -1);
                v = getRoot(v, -1, siz[v]);
                // pdt[u].push_back(v);
                build(v);
            }
        }
    }
    PointerDivide(vector<vector<pair<int, int>>> &g) : g(g), siz(g.size()), vis(g.size())
    {
        dfsSize(1, 0);
        root = getRoot(1, 0, siz[1]);
        build(root);
    }
};
```

### 0.11 树上启发式合并

这是一个离线做法，本质就是对于每个节点都通过 $DFS$ 求答案，但是通过保留重儿子的答案使复杂度降为优秀的 $nlogn$

具体做法是，对于以 $u$ 为根的子树，我们先递归其轻儿子，但是信息我们不做保留，注意这次 $dfs$ 并不是在求轻儿子的对 $u$ 结点的答案，而是递归轻儿子求其自己的答案，递归结束后回到当前节点，我们再递归其重儿子，标记重儿子并且保留信息，然后我们开始统计该节点 $u$ 的答案，由于重儿子的信息被保留，我们只需要计算轻儿子的贡献即可，这里实际上加上了轻儿子的贡献，统计完后在此更新该节点的答案，如果该节点 $u$ 不是其父节点的重儿子，还需要清除贡献, 清除贡献前置重儿子为空，这样会把该节点轻重儿子的贡献全部删去，保证了正确性

```cpp
// 对于以u为根的子树
// 1先统计其轻儿树的答案 统计完后删除信息
// 2再统计其重子树的答案 统计完后保留信息
// 3将重子树的信息合并到u上
// 4再遍历u的轻子树 把轻子树的信息合并到u上
// 5判断u的信息是否需要传给父节点（如果u是其父节点的重儿子就需要传递信息）

// 以求子树颜色种类为例
template <class T>
struct DSUOnTree{
    int n;
    vector<vector<pair<int, int>>> g; // 邻接表
    vector<int> siz, son;
    vector<T> ans; // 子树大小、重儿子,每个节点的答案
    int root;
    int heavyNode = 0;              // 当前处理的重儿子节点
    int typeCnt = 0;                // 颜色种类数量
    vector<int> col, colorCnt; // 原始颜色数组  颜色出现次数数组
    DSUOnTree(vector<vector<pair<int, int>>> &g, vector<int> &col, int root)
        : n(g.size() - 1), g(g), siz(n + 1), son(n + 1), ans(n + 1),
          root(root), col(col), colorCnt(n + 1, 0) {}
    // 预处理重儿子
    void preProcess(int u, int f){
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if (v != f){
                preProcess(v, u);
                siz[u] += siz[v];
                if (siz[son[u]] < siz[v]){
                    son[u] = v;
                }
            }
        }
    }
    // 添加/删除节点信息（需根据题目实现）
    // val 传 1 时是添加，传 -1 是删除
    void update(int u, int f, int val){
        // 示例：统计子树颜色种类数
        if (val == 1)
        {
            colorCnt[col[u]]++;
            typeCnt += (colorCnt[col[u]] == 1);
        }
        else
        {
            colorCnt[col[u]]--;
            typeCnt -= (colorCnt[col[u]] == 0);
        }
    }
    // 遍历子树统计/清除信息
    void traverse(int u, int f, int val){
        update(u, f, val);
        for (auto [v, w] : g[u]){
            if (v != f && v != heavyNode){
                traverse(v, u, val);
            }
        }
    }
    // 主算法
    void solve(int u, int f, bool keep){
        // 1. 先处理轻儿子（不保留信息）
        for (auto [v, w] : g[u]){
            if (v != f && v != son[u]){
                solve(v, u, false);
            }
        }
        // 2. 处理重儿子（保留信息）
        if (son[u]){
            solve(son[u], u, true);
            heavyNode = son[u];
        }
```

```
        // 3. 再次遍历轻儿子并合并信息
        for (auto [v, w] : g[u]){
            if (v != f && v != son[u]) traverse(v, u, 1);
        }
        update(u, f, 1); // 添加当前节点信息
        // 4. 此处可保存当前节点的答案
        ans[u] = typeCnt;
        heavyNode = 0;
        // 5. 如果需要清除信息
        if (!keep){
            traverse(u, f, -1);
            typeCnt = 0;
        }
    }
    // 对外接口
    void compute()
    {
        preProcess(root, 0); // 假设根节点是1
        solve(root, 0, true);
    }
};
```

## 0.12 基环树与基环树森林

基环树又名环套树，是 $n$ 个点 $n$ 条边的无向连通图，我们看待基环树的思路往往是环和树分开，这个思想是处理基环树的关键，也就是说，基环树实际上是一个环，且环上每个点都悬挂着一颗以该点为根的子树，如果我们破环成链，那么形状将是一条长链，链上每个节点都有一颗子树 (可能为空)

我们往往对基环树问题有两种处理方法

1 对于在基环树上贪心的问题，我们往往先在每棵小树上 $dfs$ 合并信息到环上，然后再对环单独处理。

2 对于基环树一些性质的题目，以基环树直径为例，我们往往要破环成链，分类讨论。具体的说，我们任选环上某条边记为 $(u,v)$, 我们将其断开，那么剩下的结构将是一棵树，我们可以朴素的求出它的直径，这是不经过 $(u,v)$ 边的最大直径，然后我们考虑一定经过 $(u,v)$ 边的最大直径，我们不难发现，直径一定是破环后的链的一个前缀 + 一个后缀 + 该边拼凑而成，我们要先对每棵小树处理出它的最大深度 $dep$，那么 $pre[u]=max(pre[u-1],dep[u]+len[u])$，$len[u]$ 是链左边端点到该点的长度，我们求出前后缀 $max$ 后，遍历取最大即可

**基环树**

```
struct RingTree{
    vector<vector<pair<int, int>>> g; // 基环树上的小树
    vector<int> ring, ringVal;        // 环,环的出点的权值
    // ringVal[i]对应的是 ring[i]和ring[i+1]的权值

    RingTree(vector<vector<pair<int, int>>> e) : g(e.size()){
        int n = e.size() - 1;
        vector<int> vis(n + 1, 0), deg(n + 1, 0);
        for (int u = 1; u <= n; ++u){
            for (auto [v, w] : e[u]){
                deg[v]++;
            }
        }
        auto dfs1 = [&](auto dfs1, int u) -> void{
            vis[u] = 1;
            for (auto [v, w] : e[u]){
                if (!vis[v]){
                    // dsu.merge(u, v);
                    g[v].emplace_back(u, w);
                    deg[v]--;
                    if (deg[v] == 1){
                        dfs1(dfs1, v);
                    }
                }
            }
        };
        for (int i = 1; i <= n; ++i){
            if (!vis[i] and deg[i] == 1) // 从叶子开始剥离树的结构
            {
                dfs1(dfs1, i);
            }
        }
        auto dfs2 = [&](auto dfs2, int u) -> void{
            ring.push_back(u);
            vis[u] = 1;
            for (auto [v, w] : e[u]){
                if (!vis[v]){
                    ringVal.push_back(w);
                    dfs2(dfs2, v);
                }
            }
        };
```

```
        for (int i = 1; i <= n; ++i){
            if (!vis[i]) // 一定是环点
            {
                dfs2(dfs2, i); // 找到整个环
                for (auto [v, w] : e[ring.back()]){
                    if (v == ring.front()){
                        ringVal.push_back(w); // 把环起点和终点的边权加入
                        break;
                    }
                }
            }
        }
    }
};
```

基环树森林是一个更特殊的情况，它往往比基环树更加常见，其题目往往有特征为每个点只有唯一的入点或唯一的出点，也就是说我们可以理解为，在树上它有唯一的父亲，那么如果我们连边 $i - fa[i]$，将会得到一个基环树森林，基环树的处理手法和基环树如出一辙，无非就是对多棵树处理。

注意有些时候题目可能不是单纯的基环树森林，它可能是树森林和基环树森林的并集，需要分类讨论

**基环树森林**

```
struct RingTreeForest{
    int cnt = 0 ,n;
    vector<int> &fa; // 基环树的森林的出边/父边
    vector<int> onRing, root, ringLen, pos, posOnRing;
    // onRing[i]表示i是否在环上，root[i]表示i所在的树的根，ringLen存所有环的长度，
    // pos[i]表示i点在哪个环上，posOnRing[i]表示i在环上的位置
    vector<vector<int>> ring, id;
    // ring[i]存环i的所有点，下标从0开始
    vector<vector<int>> g;
    RingTreeForest(vector<int> &b) : n(b.size()), fa(b), g(n),
    onRing(n, 0), root(n, 0), pos(n, 0),
    posOnRing(n, 0), ring(n){
        vector<int> vis(n, 0);
        for (int u = 1; u <= n - 1; ++u) {
            int v = u;
            while (!vis[v] and v) // 防止跳到0
            {
                vis[v] = u;
                v = fa[v];
            }
            if (vis[v] == u and v) // 跳到的不是0于是成环
            {
                ++cnt;
                int k = v, cur = 0;
                do{
                    ring[cnt].push_back(k);
                    onRing[k] = 1;
                    pos[k] = cnt;
                    root[k] = k;
                    posOnRing[k] = cur++;
                    k = fa[k];
                } while (k != v);
                ringLen.push_back(cur);
                // update a new ring
            }
        }
        for (int i = 1; i <= n - 1; ++i) {
            if (!onRing[i] and fa[i]) // 不在环上且fa不为空
            {
                g[fa[i]].push_back(i);
            }
        }
    }
};
```

## 0.13 虚树

虚树是指我们在原本树的基础上抽离出一颗新的树，虚树只由关键点以及关键点的 $LCA$ 构成，在预处理一些信息以后我们可以在虚树上求解只涉及关键点的答案。复杂度由 $O(n)$ 降为 $O(V)$，其中 $V$ 为关键点的数量。

构建方法：

因为多个点的 $LCA$ 可能是同一个点，为了保证复杂度我们不能多次加入。

我们首先将关键点集合按 $dfn$ 序排序，然后将相邻点的 $lca$ 加入点集。

之后再次按 $dfn$ 序排序并去重。然后把相邻的两个点 $(u, v)$ 连边 $lca(u, v) -> v$

```
template <class T>
struct VirtualTreePre
{
    TreePre<T> pre;
    int root;
    vector<vector<pair<int, int>>> vt;
    VirtualTreePre(vector<vector<pair<int, int>>> &g, int root)
        : pre(g, root), root(root), vt(g.size()) {}
    void build(vector<int> a)
    {
        a.push_back(root); //加入根节点方便DP,根据题目情况判断是否加root
        sort(begin(a), end(a), [&](int u, int v)
                { return pre.dfn[u] < pre.dfn[v]; });
        for (int i = a.size() - 1; i; --i)
        {
            a.push_back(pre.getLca(a[i], a[i - 1]));
        }
        sort(begin(a), end(a), [&](int u, int v)
                { return pre.dfn[u] < pre.dfn[v]; });
        a.erase(unique(begin(a), end(a)), end(a));
        for (auto u : a)
        {
            vt[u].clear();
        }
        for (int i = 1; i < a.size(); ++i)
        {
            int lca = pre.getLca(a[i], a[i - 1]);
            vt[lca].emplace_back(a[i], pre.dis[a[i]] - pre.dis[lca]);
        }
    }
};
```

## 0.14 拓扑排序

拓扑排序是处理 $DAG$ 的算法，其本质是按顺序处理每个点，处理该点时它的所有前驱点已经处理完毕，从而保证信息不会遗漏。它有如下的性质:

只有 $DAG$ 才存在拓扑序，拓扑序是一个点的有序排列，其大小等于原图总点数。拓扑序不止一种，但在没有特殊性质下，拓扑序计数没有多项式时间解，但我们可以通过改队列为优先队列求出最大/最小拓扑序，当然也能判断图是不是 $DAG$

在 $DAG$ 上跑拓扑排序 $DP$，可以解决 $DAG$ 最长路，$DAG$ 路径计数,$DAG$ 点对可达情况等等

对于 $DAG$ 点对可达情况我们可以 $bitset$ 优化, 即 $f(u)|=f(v)$

ps: 算一个 DAG 有多少种可能的 topo 序 是一个经典的 #P-完全问题（#P-complete）

```
struct Toposort{
    int n;                              // 节点数量
    std::vector<std::vector<int>> g; // 邻接表
    std::vector<int> order;          // 存储拓扑序
    // 构造函数，初始化节点数量
    Toposort(int n) : n(n), g(n + 1) {}
    // 添加有向边 u->v
    void addEdge(int u, int v){
        g[u].push_back(v);
    }
    // 执行拓扑排序，返回是否成功（无环）
    bool sort(){
        std::vector<int> deg(n + 1, 0);
        order.clear();
        order.reserve(n);
        // 计算入度
        for (int u = 1; u <= n; ++u){
            for (int v : g[u]){
                ++deg[v];
            }
        }
        std::queue<int> q;
        // 找出所有入度为0的节点
        for (int u = 1; u <= n; ++u) {
            if (deg[u] == 0)  {
                q.push(u);
            }
        }
        while (!q.empty()){
            int u = q.front();
            q.pop();
            order.push_back(u);
            // 减少相邻节点的入度
            for (int v : g[u]) {
                if (--deg[v] == 0) {
                    q.push(v);
                }
            }
        }
        // 如果拓扑序包含所有节点，则无环
        return order.size() == n;
    }
    // 获取拓扑排序结果
    const std::vector<int> &getOrder() const {
        return order;
    }
};
```

## 0.15 树哈希

树哈希主要解决树的同构问题，即快速判断两颗树的结构是否相同

首先我们得有一个哈希函数 $f$, 树哈希想要不被卡关键在于哈希函数的设置不能是线性函数 $ax+b$，我们可以设置成 $ax^3+b$，对于整棵树哈希值的计算，我们的公式是

$hash[u]=1+\sum_{v\in son(u)} f(hash[v])$

有了这两个前置知识，我们分有根树和无根树来讨论

- 有根树，这种情况十分简单，我们从指定的根开始 $dfs$，求出根节点的哈希值，如果两个根节点的哈希值相同，那么两颗有根树同构
- 无根树，对于无根树的同构，我们有两种方法。

其一是先求出树的一个或两个重心，以重心为根 $dfs$ 求出哈希值，如果有两个重心就把两个哈希值异或起来，然后对比即可，需要注意的是，如果存在两个重心，就必须求两个重心为根的哈希值，否则会出错 其二是求出以每个点为根的哈希值，然后异或起来得到最终哈希值，操作方法是朴素的换根 $DP$，模板代码给出的就是换根 $DP$ 做法

```
using u64 = unsigned long long;

template <class T>
struct Rand
{
    std::mt19937 myrand;
    Rand(const long long seed = time(0)) : myrand(seed) {}
    T operator()(T l, T r)
    {
        return std::uniform_int_distribution<T>(l, r)(myrand);
    }
};
Rand<u64> rd;

u64 f(u64 x)
{
    const static u64 r1 = rd(1 << 20, 1 << 24);
    const static u64 r2 = rd(1 << 25, 1 << 30);
    const static u64 mask = (1ll << 31) - 1;

    auto h = [&](u64 y)
    {
        return (u64)y * y * y * r1 + r2;
    };
    return h(x & mask) + h(x >> 31);
}

struct Treehash // 得到每个点为根的hash值 On
{
    int n;
    std::vector<std::vector<int>> g;
    std::vector<u64> hash;
    u64 ans = 0;
    void dfs1(int u, int fa)
    {
        hash[u] = 1;
        for (auto v : g[u])
        {
            if (v == fa)
            {
                continue;
            }
            dfs1(v, u);
            hash[u] += f(hash[v]);
        }
    }
    void dfs2(int u, int fa)
    {
        for (auto v : g[u])
        {
            if (v == fa)
            {
                continue;
            }
            hash[v] = hash[v] + f(hash[u] - f(hash[v]));
            dfs2(v, u);
        }
    }
    Treehash(std::vector<std::vector<int>> &e) : n(e.size() - 1), g(e), hash(n + 1)
    {
        dfs1(1, 0);
        dfs2(1, 0);
        for (int i = 1; i <= n; ++i)
        {
```

```
            ans ^= hash[i];
        }
    }
};
```

## 0.16 最短路

### 0.16.1 单源最短路

首先我们最短路算法的精髓主要是松弛，松弛的定义为对于一条边 $(u, v, w)$, 如果满足 $d[u] + w < d[v]$
我们让 $d[v] = d[u] + w$ 就叫松弛成功

**0.16.1.1 BFS** 由于队列先进先出的特性，在边权只有 $0/1$ 时我们可以采用 $BFS$ 求解最短路。
对于边权为 $0$ 的情况，我们放到队列的头，对于边权为 $1$ 的情况，我们放到队列的尾。

**0.16.1.2 BellmanFord** $Bellmanford$ 是暴力的松弛算法，根据松弛的定义我们知道每个点达到最优态最多需要 $n-1$ 轮，每轮松弛需要遍历所有的边松弛。特别的，如果到了第 $n$ 轮还能松弛成功的话，就说明有负环.
复杂度 $O(nm)$
但是根据算法流程，我们发现 $Bellmanford$ 算法可以计算最多经过 $k$ 条边的最短路，只需要松弛 $k$ 轮即可

```cpp
template <class T>
struct BellmanFord
{
    int n;                                          // 节点数量
    const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
    std::vector<T> dis;                             // 最短距离
    std::vector<int> pre;                           // 前驱节点

    // 构造函数，初始化节点数量
    BellmanFord(int n)
        : n(n),
          g(n + 1),
          dis(n + 1, INF),
          pre(n + 1, -1) {}

    // 添加有向边 u->v，权重为 w
    void addEdge(int u, int v, int w)
    {
        g[u].emplace_back(v, w);
    }
    // 执行 Bellman-Ford 算法
    // 参数：起点 st，可选参数 k 表示最多经过 k 条边
    [[nodiscard]] bool run(int st, int k = -1)
    {
        dis.assign(n + 1, INF);
        pre.assign(n + 1, -1);
        dis[st] = 0;

        bool cycle = false;                         // 标记负环
        int nums = (k == -1) ? n : k; // 迭代轮数

        for (int i = 1; i <= nums; ++i)
        {
            bool relaxed = false;
            auto lastDist = dis; // 保存上一轮距离，防止串联

            for (int u = 1; u <= n; ++u)
            {
                if (lastDist[u] == INF)
                    continue;
                for (const auto &[v, w] : g[u])
                {
                    if (lastDist[u] + w < dis[v])
                    {
                        dis[v] = lastDist[u] + w;
                        pre[v] = u;
                        relaxed = true;
                    }
                }
            }
            // 如果第 n 轮仍有松弛操作，说明存在负权环
            if (i == nums && relaxed && k == -1)
            {
                cycle = true;
            }
            // 如果本轮没有松弛操作，可以提前终止
            if (!relaxed)
                break;
        }
        return cycle;
```

```
    }
    // 获取最短距离数组
    const std::vector<T> &getDistance() const
    {
        return dis;
    }
};
```

**0.16.1.3 SPFA** 由于 $Bellman ford$ 的复杂度不够优秀，我们引入它的优化版 $SPFA$，我们发现每轮松弛有很多没必要的状态被操作，这大大浪费了时间.

所以 $SPFA$ 算法在 $Bellman ford$ 的基础上采用了一个队列，队列里记录了可能会松弛成功的点，加快了算法的执行速度。同样 $SPFA$ 也能判断负环，我们用一个 $cnt$ 记录松弛次数即可

作为处理负环的最短路算法, 在一般的图上十分高效, 但仍可以被毒瘤数据卡成 $O(nm)$ 的复杂度, 尽管如此 $SPFA$ 仍然是我们在面对负数边权时最好的选择

```cpp
template <class T>
class SPFA
{
private:
    int n;                                          // 节点数量
    const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
    std::vector<T> dis;                             // 最短距离
    std::vector<int> pre;                           // 前驱节点
    std::vector<int> inQueue;                       // 节点是否在队列中
    std::vector<int> relaxCount;                    // 节点松弛次数

public:
    // 构造函数，初始化节点数量
    SPFA(int n)
        : n(n),
          g(n + 1),
          dis(n + 1, INF),
          pre(n + 1, -1),
          inQueue(n + 1, 0),
          relaxCount(n + 1, 0) {}

    // 添加有向边 u->v，权重为 w
    void addEdge(int u, int v, int w)
    {
        g[u].emplace_back(v, w);
    }

    // 执行 SPFA 算法
    // 参数：起点 st
    // 返回值：pair<bool, bool>
    //   first: 是否成功找到最短路径
    //   second: 是否存在负权环
    [[nodiscard]] std::pair<bool, bool> run(int st)
    {
        // 初始化
        dis.assign(n + 1, INF);
        pre.assign(n + 1, -1);
        inQueue.assign(n + 1, 0);
        relaxCount.assign(n + 1, 0);
        dis[st] = 0;
        std::queue<int> q;
        q.push(st);
        inQueue[st] = 1;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            inQueue[u] = 0;

            for (const auto &[v, w] : g[u])
            {
                if (dis[u] + w < dis[v])
                {
                    dis[v] = dis[u] + w;
                    pre[v] = u;
                    relaxCount[v] = relaxCount[u] + 1;

                    // 如果松弛次数超过n次，说明存在负权环
                    if (relaxCount[v] >= n)
                    {
                        return {false, true};
                    }

                    if (!inQueue[v])
                    {
                        q.push(v);
                        inQueue[v] = 1;
                    }
                }
            }
```

```
        }

        return {true, false};
    }

    // 获取最短距离数组
    const std::vector<T> &getDistance() const
    {
        return dis;
    }

    // 获取从起点到终点的最短路径
    std::vector<int> getPath(int target) const
    {
        std::vector<int> path;
        if (pre[target] == -1 && dis[target] == INF)
        {
            return path; // 不可达
        }

        for (int u = target; u != -1; u = pre[u])
        {
            path.push_back(u);
        }
        std::reverse(path.begin(), path.end());
        return path;
    }

    // 检查图中是否存在负权环
    bool has_negative_cycle()
    {
        // 需要重新初始化
        dis.assign(n + 1, 0);     // 使用0初始化以检测所有可能的负环
        inQueue.assign(n + 1, 1); // 初始所有节点都在队列中
        relaxCount.assign(n + 1, 0);

        std::queue<int> q;
        for (int i = 1; i <= n; ++i)
        {
            q.push(i);
        }

        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            inQueue[u] = 0;

            for (const auto &[v, w] : g[u])
            {
                if (dis[u] + w < dis[v])
                {
                    dis[v] = dis[u] + w;
                    relaxCount[v] = relaxCount[u] + 1;

                    if (relaxCount[v] >= n)
                    {
                        return true;
                    }

                    if (!inQueue[v])
                    {
                        q.push(v);
                        inQueue[v] = 1;
                    }
                }
            }
        }
        return false;
    }
};
```

#### 0.16.1.4 Dijkstra

Dijkstra 是我们最为常用的最短路算法之一，可以处理无负边图的最短路，常规堆优化的最短路复杂度为 $O(mlogm)$，如果我们使用 $pbds$ 中的斐波那契堆优化，那么复杂度将是 $O(nlogn+m)$

特别的，当边权只有 $0$ 和 $1$ 时, 我们使用 $BFS$ 求解最短路

由于 $pbds$ 中的配对堆不支持配模板，因此该板子没写类模板封装，需要手动改数据类型。

```
struct Dijkstra{
    using Edge = std::pair<int, int>; // first: 目标节点, second: 边权重
    // 自定义比较函数，按距离排序
    struct Compare{
        bool operator()(const Edge &a, const Edge &b) const{
            return a.second > b.second; // 小根堆
        }
    };
    // 使用配对堆（最接近斐波那契堆的实现）
    using Heap = __gnu_pbds::priority_queue<
        Edge,
        Compare,
        __gnu_pbds::pairing_heap_tag>;
    std::vector<int> dis;
    const int INF = std::numeric_limits<int>::max() / 2;
    int n;
    // 初始化
    Dijkstra(int n) : n(n), dis(n + 1) {}
    // 执行Dijkstra算法
    void run(int S, const std::vector<std::vector<Edge>> &g){
        dis.assign(n + 1, INF);
        std::vector<bool> vis(n + 1, false);
        std::vector<Heap::point_iterator> handles(n + 1);
        Heap pq;
        dis[S] = 0;
        handles[S] = pq.push({S, 0});
        while (!pq.empty()){
            auto [u, d] = pq.top();
            pq.pop();
            if (vis[u]) continue;
            vis[u] = true;
            for (const auto &[v, w] : g[u]){
                if (!vis[v] && dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    if (handles[v] != nullptr){
                        pq.modify(handles[v], {v, dis[v]});
                    }else{
                        handles[v] = pq.push({v, dis[v]});
                    }
                }
            }
        }
    }
    // 获取结果
    const std::vector<int> &getDistance() const{
        return dis;
    }
};
```

还能不能优化呢？如果还要追求更极致的效率，需要发掘一些性质

比如：如果边权总和有限/最短路不会很长，我们可以用桶配合距离实现最小堆的功能，这样可以做到 $O(n+m)$

**0.16.1.5 最短路计数** 我们不妨设源点为 $1$，我们要求源点到其他点最短路的条数，考虑 $dijkstra$ 的过程，我们开一个 $cnt$ 数组记录最短路条数, 如果 $d[u]+w<d[v]$，那么 $d[u]+w$ 将成为新的最短路, 所以我们让 $cnt[v]=cnt[u]$, 而如果是 $d[u]+w=d[v]$, 那么 $v$ 的最短路将会更多, 我们让 $cnt[v]=cnt[v]+cnt[u]$
注意先判 $vis$, 无论是大于还是等于都要入队

**0.16.1.6 差分约束** 差分约束用来求解一形如 $x1\leq x2+c$ 的不等式组的可行解问题，我们通过建立有向图来解决
我们把每个未知数转化为图上的每个点, 我们建立虚拟源点 0, 向每个点连边权为 0 的有向边。
对于一个未知数 $x$, 从超级源点到它的距离就是一个可行解
我们接下来详细介绍如何建图，分求最大解和最小解两种情况
$1)$ 求最大可行解
未知数 $x$ 的最大可行解为，从源点到 $x$ 的最短路 $dis(x)$
首先我们将所有不等式转化为 $x1\leq x2+C$ 我们连边 $x2->x1$ 边权为 $C$
如果不等式形如 $x1<x2+C$ 我们转化为 $x1\leq x2+C-1$
如果不等式形如 $C\leq x$ 我们转化为 $x0\leq x-C$ 其中 $x0$ 为超级源点
$2)$ 求最小可行解
未知数 $x$ 的最小可行解为，从源点到 $x$ 的最长路 $dis(x)$
首先我们将所有不等式转化为 $x1+C\leq x2$ 我们连边 $x2->x1$ 边权为 $C$
如果不等式形如 $x1+C<x2$ 我们转化为 $x1+C+1\leq x2$
如果不等式形如 $x\leq C$ 我们转化为 $x-C\leq x0$ 其中 $x0$ 为超级源点
我们选择合适的最短路算法求解即可 如果存在负环，那么原不等式组无解（一般用 SPFA）

**0.16.1.7 分层图最短路** 分层图最短路是将原本的一层图复制变成多层图帮助我们求解答案的一个 $Trick$，题目往往形如，对一个边带权图 $G=(V,E)$, 要求 $S-T$ 的最短路，但是我们可以免费走 $K$ 条边，我们复制 $K$ 层相同的图，在层与层之间连边权为 $0$ 的边，这样就达到了免费走边的目的，如果当前在第 $i$ 层图，相当于走了 $i-1$ 条免费边

**0.16.1.8 最短路树和最短路 DAG** 最短路树:
在我们求解以 $S$ 为起点的单源最短路时, 我们每次对一个点成功松弛时, 即 $d[u]+w->d[v]$ 时, 我们令 $fa[v]=u$, 也就是说我们记录每个点的前驱, 最终会形成一棵树的结构, 称作最短路树. 最短路树时处理变形最短路问题的关键方法.
最短路 DAG:
我们求解完单源最短路径之后, 假设起点为 $S$，我们建立一张新图，我们对于每条边 $u->[v,w]$, 如果有 $dis[u]+w=dis[v]$, 那么我们在新图上连边 $u->v$，这样我们将得到一张 $DAG$, 对于这张 $DAG$ 的所有生成树，都是一个合法的最短路树，因此我们可以套用矩阵树定理求解最短路树数量

#### 0.16.1.9 删边/带修最短路

这是一个非常经典的问题, 题目形式为 $q$ 次询问，每次询问删去某条边 $S-T$ 的最短路, 删除非永久化. 每次询问的 $S-T$ 相同.

我们用 $path(x,y)$ 表示最短路树上 $(x,y)$ 的路径 $D$ 表示原最短路长度

我们先求出 $path(S,T)$, 注意到 $path(S,T)$ 是一段连续的边的区间

如果询问删去的边不属于 $path(S,T)$，那么 $ans=D$

如果询问删去的边属于 $path(S,T)$, 我们预处理出每条非最短路边的贡献，对于一条非最短路边 $u->[v,w]$，我们把经过这条边的最短路表示为 $dis(S,u)+w(u,v)+dis(v,T)$ 不难发现 $w(u,v)$ 有贡献的区间是 $[L,R]$, 其中 $L$ 为 $path(S,T)$ 和 $path(S,u)$ 第一个非重合点的位置，$R$ 为 $path(T,S)$ 和 $path(T,v)$ 第一个非重合点的位置。我们可以通过最短路树上 $DP$ 求出所有点的 $L,R$, 这里的贡献的含义是，删去这个区间的边后我可以用 $w(u,v)$ 代替, 因此我们使用区间 $min$ 线段树直接维护最短路值 $dis(S,u)+w(u,v)+dis(v,T)$, 每次询问单点查询即可

```
template <class G>
class EdgeDeleteShortestPath
{
public:
    SegmentTree<G> *Tree;//区间min线段树
    const G inf = std::numeric_limits<G>::max() / 2;//上限
    std::map<std::pair<int, int>, int> mp;
    std::vector<G> disS, disT;//S为起点的dis数组  T为起点的dis数组
    std::vector<int> preS, preT;//
    G before;//before表示最开始的最短路
    EdgeDeleteShortestPath(int S, int T, std::vector<std::vector<std::pair<int, G>>> &g)
    {
        int n = g.size() - 1;
        Dijkstra dijk(n);
        dijk.run(S, g);
        before = dijk.dis[T];
        preS = dijk.getPre();
        disS = dijk.getDistance();
        dijk.run(T, g);
        preT = dijk.getPre();
        disT = dijk.getDistance();
        std::vector<int> path; // 从s->t
        int now = S;
        while (now != -1){
            path.push_back(now);
            now = preT[now];
            if (path.size() >= 2){
                mp[{path[path.size() - 2], path.back()}] = path.size() - 1;
                mp[{path.back(), path[path.size() - 2]}] = path.size() - 1; // 记录路径是第几条边
            }
        }
        std::vector<std::vector<int>> treeS(n + 1), treeT(n + 1); // 最短路树
        for (int i = 1; i <= n; ++i){
            if (preS[i] != -1) {
                treeS[preS[i]].push_back(i);
            }
            if (preT[i] != -1){
                treeT[preT[i]].push_back(i);
            }
        }
        int PathSum = path.size() - 1;
        std::vector<int> dpS(n + 1, 0), dpT(n + 1, 0);
        auto dfsS = [&](auto dfsS, int u) -> void{
            for (auto v : treeS[u]){
                if (mp.contains(std::make_pair(u, v))) // 说明还在干线上
                {
                    dpS[v] = mp[{std::make_pair(u, v)}] + 1;
                    dfsS(dfsS, v);
                }else{
                    dpS[v] = dpS[u];
                    dfsS(dfsS, v);
                }
            }
        };
        dpS[S] = 1;
        dfsS(dfsS, S);
        auto dfsT = [&](auto dfsT, int u) -> void{
            for (auto v : treeT[u]){
                if (mp.contains(std::make_pair(u, v))) // 说明还在干线上
                {
                    dpT[v] = mp[{std::make_pair(u, v)}] - 1;
                    dfsT(dfsT, v);
                }else{
                    dpT[v] = dpT[u];
                    dfsT(dfsT, v);
```

```
                }
            }
        };
        dpT[T] = path.size() - 1;
        dfsT(dfsT, T);
        std::vector<G> cur(path.size(), inf);
        Tree = new SegmentTree<G>(cur);
        for (int u = 1; u <= n; ++u){
            for (auto [v, w] : g[u]){
                if (!mp.contains({u, v})) // 说明不是最短路边需要更新答案
                {
                    if (dpS[u] <= dpT[v] and dpT[v] <= path.size() - 1 and dpS[u] >= 1)
                    {
                        Tree->rangeGetMin(dpS[u], dpT[v], disS[u] + disT[v] + w);
                    }
                }
            }
        }
    }
    G getShortestPath(int u, int v){
        if (!mp.contains({u, v})){
            return before;
        }else{
            return Tree->rangeQuerySum(mp[{u, v}], mp[{u, v}]);
        }
    }
};
```

#### 0.16.1.10 平面图最小割

我们有如下定理:

对于一个平面图来说，$S-T$ 的最小割等价于在其对偶图上的最短路。

实际上我们最大的困难在于把平面图转化为其对偶图

#### 0.16.1.11 同余最短路

当出现形如给定 $n$ 个整数，求这 $n$ 个整数能拼凑出多少个其他整数（$n$ 个整数可以重复取），以及给定 $n$ 个整数，求这 $n$ 个整数不能拼凑出的最小 (最大) 的整数，或者至少要拼几次才能拼出模 $P$ 余 $K$ 的数的问题时可以使用同余最短路的方法。

我们用例题来说明，给你 $x,y,z$，请你求出 $ax+by+cz(a,b,c\geq 0)$ 能构成 $[1,h]$ 区间内多少个不同的数字?

我们不妨设 $x<y<z$，我们考虑不使用最小数字 $x$ 的情况下，求出该表达式 $modx=i$ 的最小数字，具体的说，假如余数为 $i$ 的最小数为 $d_i$, 那么所有的 $d_i+kx$ 都能构成。那么怎么求出最小数呢，我们可以进行的操作为 $i->(i+y)modx$ 和 $i->(i+z)modx$，不难发现这是一个有向边，边权分别为 $y$ 和 $z$，这启发我们使用最短路算法得到所有的 $d_i$, 那么最终答案为 $\sum_{i=0}^{x-1}((h-d_i)/x+1)$, 注意源点为 $0$ 且

```
void solve(){
    ull h;cin>>h;
    ull x,y,z;cin>>x>>y>>z;
    if(x>y) swap(x, y);
    if(x>z) swap(x, z);
    vector<ull> dis(x,LONG_LONG_MAX);
    priority_queue<ull,vector<ull>,greater<>> q;q.push(1);
    dis[1%x] = 1;
    while(q.size()){
        auto v = q.top();q.pop();
        if(v>dis[v%x]) continue;
        if((i128)v+y<=h and v+y<dis[(v+y)%x]) {
            q.push(v+y);
            dis[(v+y)%x] = v+y;
        }
        if((i128)v+z<=h and v+z<dis[(v+z)%x]) {
            q.push(v+z);
            dis[(v+z)%x] = v+z;
        }
    }
    ull ans = 0;
    for(int i = 0;i<x;i++){
        if(dis[i]>h) continue;
        ans += 1+(h-dis[i])/x;
    }
    cout<<ans<<"\n";
}
```

### 0.16.2 全源最短路

全源最短路实际上是求出图上任意两点间的最短路。
我们分稠密图和稀疏图来分别考虑
对于稠密图而言，$Floyed$ 算法是最好的选择，因为时间复杂度与 $m$ 无关，我们可以在 $O(n^3)$ 的时间内解决该问题
对于稀疏图来说，我们有两种选择，对于没有负边的图，我们可以跑 $n$ 次 $dijkstra$ 算法来实现，复杂度 $n^2logn$, 对于有负权的图，我们只能采用全源最短路算法 $johnson$ 算法，复杂度为 $nmlogn$

#### 0.16.2.1 Floyd

```cpp
template <class T>
class Floyd
{
private:
    int n;                                          // 节点数量
    const T INF = std::numeric_limits<T>::max() / 2; // 防止加法溢出
    std::vector<std::vector<T>> dp;                 // 距离矩阵
    std::vector<std::vector<int>> next;             // 路径重建矩阵

public:
    // 构造函数，初始化节点数量
    Floyd(int n)
        : n(n),
          dp(n + 1, std::vector<T>(n + 1, INF)),
          next(n + 1, std::vector<int>(n + 1, -1))
    {

        // 初始化对角线为0
        for (int i = 1; i <= n; ++i)
        {
            dp[i][i] = 0;
            next[i][i] = i;
        }
    }

    // 添加无向边 u-v，权重为 w（自动处理重边）
    void addEdge(int u, int v, T w){
        dp[u][v] = std::min(dp[u][v], w);
        dp[v][u] = std::min(dp[v][u], w);
        next[u][v] = v;
        next[v][u] = u;
    }
    // 执行 Floyd-Warshall 算法
    void run()
    {
        for (int k = 1; k <= n; ++k)
        {
            for (int i = 1; i <= n; ++i)
            {
                for (int j = 1; j <= n; ++j)
                {
                    if (dp[i][k] + dp[k][j] < dp[i][j])
                    {
                        dp[i][j] = dp[i][k] + dp[k][j];
                        next[i][j] = next[i][k];
                    }
                }
            }
        }
    }
    // 获取距离矩阵
    const std::vector<std::vector<T>> &getDistanceMatrix() const
    {
        return dp;
    }
    // 获取两点间的最短距离
    T getDistance(int u, int v) const
    {
        return dp[u][v];
    }
    // 获取两点间的最短路径
    std::vector<int> getPath(int u, int v) const
    {
        std::vector<int> path;

        // 不可达情况
        if (next[u][v] == -1)
        {
```

```
            return path;
        }
        path.push_back(u);
        while (u != v)
        {
            u = next[u][v];
            path.push_back(u);
        }
        return path;
    }
    // 检查图中是否存在负权环
    bool hasNegativeCycle() const
    {
        for (int i = 1; i <= n; ++i)
        {
            if (dp[i][i] < 0)
            {
                return true;
            }
        }
        return false;
    }
};
```

#### 0.16.2.2 Johnson

$Johnson$ 算法实际上也是跑 $n$ 轮 $dijkstra$ 算法的做法，但我们提前处理了负边权, 方法如下

我们建立超级虚拟源点 $0$，向每个点连接边权为 $0$ 的有向边，然后我们跑 $SPFA$ 算法得到 $0$ 向每个点的最短路 $D(i)$，然后我们重新设定边权，对原先的边 $(u,v,w)$, 我们改为 $(u,v,w+d[u]-d[v])$, 可以证明这种操作可以使新的边权为非负数, 然后我们利用新边权跑 $n$ 轮 $dijkstra$ 算法，最终两条边实际的最短路 $Dis(u,v)=dis(u,v)+d[u]-d[v]$, 其中 $dis(u,v)$ 是 $dijkstra$ 的结果

实现完全基于 $SPFA$ 和 $Dijkstra$ 算法，在此不给出代码

#### 0.16.2.3 汇总

```
//变形
//1.最长路。把所有边取相反数即可
//2.乘积最大，边权相乘，可以通过取对数转化为加法。
//3.如果是单向边，求所有点到某一个的距离，可以反向建边，然后跑dijkstra
//4.分层最短路
//1.建图时直接建成k+1层。
//2.多开一维记录机会信息。
//5.并查集配合dijkstra一遍，可求出对于特定的点集，每个点最近的那个同属于点集点的距离，
//6.差分约束系统
//若有 Xi-Xj<=Ck，则连一条有向边 j->i，边权为 Ck。这样求最短路可以保证 Xi-Xj<=Ck
//        然后再添加一个 0 号节点，向其他节点连一条边权为 0 的有向边，表示 Xi-X0<=0
//这样，在无负环时会得到一组非正数解（先求一组解，然后再全部加上 Δ）使用spfa可以判负环
//        最短路求得是最大值，最长路求得是最小值。(w(a,b) 表示任意一条a到达b的路径 )

template<typename T>
struct Shortest_path {
    int n;
    T inf;
    std::vector<T> dis, cnt;
    std::vector<std::vector<T>> d;
    std::vector<std::vector<std::pair<int, T>>> adj, e;
    std::vector<int> vis;
    std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int>>, std::greater<>> pq;

    Shortest_path() {};

    Shortest_path(int _n, T _d = 1e9) : n(_n), inf(_d) {
        Init(_n);
    }

    void Init(int _n) {
        dis.assign(_n, inf);
        cnt.assign(_n, 0);
        vis.assign(_n, false);
        adj.assign(_n, {});
    }

    void addEdge1(int u, int v, T w) {
        adj[u].emplace_back(v, w);
    }

    void addEdge2(int u, int v, T w) {
        adj[u].emplace_back(v, w);
        adj[v].emplace_back(u, w);
    }

    void dijkstra(int s = 1) {
        fill(dis.begin(), dis.end(), inf);
        fill(vis.begin(), vis.end(), -1);

        pq.emplace(0, s);
        dis[s] = 0;
        while (!pq.empty()) {
            int u = pq.top().second;
            pq.pop();
            if (vis[u] >= 0) continue;
            vis[u] = s == u ? u + 1 : -vis[u];
            for (auto [v, w]: adj[u]) {
                if (dis[v] > 1ll * dis[u] + w) {
                    dis[v] = dis[u] + w;
                    vis[v] = -u - 1;
                    pq.emplace(dis[v], v);
                }
            }
        }
    }

    std::tuple<bool, std::vector<int>> path(int t) {
        std::vector<int> path;
        while (vis[t] - 1 != t) {
            path.emplace_back(t);
            t = vis[t] - 1;
            if (t < 0) return {false, {}};
        }
        path.emplace_back(t);
        std::reverse(path.begin(), path.end());
        return {{true}, path};
    }
```

```
    bool spfa(int s = 1) {
        std::queue<int> q;
        fill(dis.begin(), dis.end(), inf);
        fill(vis.begin(), vis.end(), 0);
        fill(cnt.begin(), cnt.end(), 0);
        q.emplace(s);
        dis[s] = 0;
        vis[s] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            vis[u] = false;
            for (auto &[v, w]: adj[u]) {
                if (dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    cnt[v] = cnt[u] + 1;
                    if (cnt[v] >= n) return true;
                    if (!vis[v]) q.push(v), vis[v] = true;
                }
            }
        }
        return false;
    }

    bool johnson() {//全源最短路O(nmlog(m))
        d.assign(n, std::vector<T>(n, inf));
        for (int i = 1; i < n; i++) addEdge1(0, i, 0);//加虚拟边

        if (spfa(0)) return false;
        auto h = dis;//一会要用，所以保存一下
        for (int u = 1; u < n; u++) {
            for (auto &[v, w]: adj[u]) {
                w += h[u] - h[v];//新边=旧边+h[u]-h[v]
            }
        }
        for (int u = 1; u < n; u++) {
            dijkstra(u);
            for (int v = 1; v < n; v++) {
                d[u][v] = dis[v] + h[v] - h[u];//原距离等于新距离+h[v]-h[u];
            }
        }
        return true;
    }

    T K_th(int s, int t, int k) {//使用A*算法
        fill(cnt.begin(), cnt.end(), 0);
        if (e.empty()) {
            e.assign(n, {});
            for (int i = 1; i < n; i++) {
                for (auto [v, w]: adj[i]) {
                    e[v].emplace_back(i, w);
                }
            }
            std::swap(e, adj);
            dijkstra(t);
        }
        if (s == t) k++;
        std::priority_queue<std::tuple<T, int, T>, std::vector<std::tuple<T, int, T>>, std::greater<>> q;//总长,u,已有的w
        q.emplace(dis[s], s, 0);
        while (!q.empty()) {
            auto [f, u, dd] = q.top();
            q.pop();
            cnt[u]++;//记录出队次数
            if (cnt[t] == k) return dd;
            for (auto [v, w]: e[u]) {
                if (cnt[v] < k) q.push({dis[v] + dd + w, v, dd + w});
            }
        }
        return -1;//没有第k短路
    }
};
```

#### 0.16.2.4 无向图最小环

我们不难想到有朴素的 $dijkstrra$ 算法，即对每条边 $(u,v,w)$, 我求出删去该边后 $u$ 到 $v$ 的最短路 $dis(u,v)$, 然后此时的最小环为 $dis(u,v)+w$，对每条边处理答案取小即可, 但复杂度十分爆炸，高达 $m^2logm$

于是我们考虑改进 $floyd$ 算法，我们注意到 $floyd$ 算法的执行过程，当外层枚举到 $k$ 时，此时所有点之间的最短路只经过 $[1,k)$ 区间内的点。我们记 $mp[u][v]$ 为原始边权，$dis[u][v]$ 为最短路，我们不难想到外层枚举到 $k$ 时，处理经过点 $k$ 的环路，其中 $k$ 是最大点，于是我们双层循环枚举 $i,j$，$i,j$ 分别为 $k$ 的两个邻点，取环路长度为 $dis[i][j]+mp[k][i]+mp[j][k]$, 然后正常跑剩下的 $floyd$ 算法

```
template <class T>
class FloydMinCycle
{
private:
    int n;                                          // 节点数量
    const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
    std::vector<std::vector<T>> dist;               // 最短距离矩阵
    std::vector<std::vector<T>> g;                  // 原始图邻接矩阵
    T mincycle;                                     // 最小环长度

public:
    // 构造函数，初始化节点数量
    FloydMinCycle(int n)
        : n(n),
          dist(n + 1, std::vector<T>(n + 1, INF)),
          g(n + 1, std::vector<T>(n + 1, INF)),
          mincycle(INF)
    {

        // 初始化对角线为0
        for (int i = 1; i <= n; ++i)
        {
            dist[i][i] = g[i][i] = 0;
        }
    }

    // 添加无向边 u-v，权重为 w（自动处理重边）
    void addEdge(int u, int v, T w)
    {
        g[u][v] = g[v][u] = std::min(g[u][v], w);
        dist[u][v] = dist[v][u] = g[u][v];
    }

    // 执行 Floyd 最小环算法
    void findMinCycle()
    {
        mincycle = INF;
        for (int k = 1; k <= n; ++k)
        {
            // 检查以k为最大节点的环
            for (int i = 1; i < k; ++i)
            {
                for (int j = i + 1; j < k; ++j)
                {
                    // 防止三个INF相加溢出
                    if (dist[i][j] < INF && g[i][k] < INF && g[k][j] < INF)
                    {
                        mincycle = std::min(
                            mincycle,
                            dist[i][j] + g[i][k] + g[k][j]);
                    }
                }
            }
            // 更新最短路径
            for (int i = 1; i <= n; ++i)
            {
                for (int j = 1; j <= n; ++j)
                {
                    if (dist[i][k] < INF && dist[k][j] < INF)
                    {
                        dist[i][j] = std::min(
                            dist[i][j],
                            dist[i][k] + dist[k][j]);
                    }
                }
            }
        }
    }

    // 获取最小环长度
    T getMinCycleLength() const
    {
```

```
        return mincycle;
    }

    // 检查图中是否存在环
    bool hasCycle() const
    {
        return mincycle < INF;
    }
};
```

#### 0.16.2.5 有向图最小平均值环

```
// 点标号为 1,2,...,n, 0为虚拟源点向其他点连权值为0的单向边.
// f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
ll f[N][N] = {Inf};
int u[M], v[M], w[M];
f[0][0] = 0;
for (int i = 1; i < n + 1; i++)
    for (int j = 0; j < m; j++)
        f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);

double ans = Inf;
for (int i = 1; i <= n; i++)
{
    double t = -Inf;
    for (int j = 1; j < n; j++)
        t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
    ans = min(t, ans);
}
```

### 0.16.3 多源最短路

这里的多源是指有多个起点，对于非源点的节点，我们只需求出到其最近源点的距离，称为多源最短路问题
朴素的最短路算法在多源最短路仍然使用，以 $dijkstra$ 算法为例，我们只需要把初始的多个源点都设置 $d(u) = 0$, 并全部 $push$ 到优先队列中即可，然后跑正常的 $dijkstra$ 算法, 同理我们也有多源 $BFS$ 算法

### 0.16.4 最长路

我们分情况考虑，如果图是一张 $DAG$, 我们在 $DAG$ 上拓扑排序朴素 $DP$ 即可，复杂度 $O(n + m)$
但如果是一张普通图，由于 $dijkstra$ 的特性只能处理最短路，我们考虑用 $SPFA$ 来解决这个问题, 操作方法很简单，只需要边权取反跑最短路即可，如果存在负环，说明最长路无穷大, 复杂度最坏

## 0.17 生成树

生成树问题是在一个无向图 $G=(V,E)$ 中找到 $V-1$ 条边，并且这 $V-1$ 条边满足联通这 $V$ 个点，然后我们会得到一个树形结构称为生成树. 常见问题有最小生成树，次小生成树等

### 0.17.1 最小生成树

我们来讨论最小生成树问题，有三种主流算法 $kruskal, prim, boruvka$
我们仍然分稠密图和稀疏图来考虑
在稀疏图中，$kruskal$ 拥有 $mlogm$ 的复杂度，相比于另外二者压倒式的好用。
其思想基于按边权排序，贪心的选, 正确性显然

#### 0.17.1.1 Kruskal

```cpp
template <class T>
struct Kruskal
{
    int n;
    int comp;
    DSU dsu;
    std::vector<std::array<int, 3>> g;

    Kruskal(int n) : n(n), dsu(n), comp(n) {}

    void addEdge(int u, int v, int w)
    {
        g.push_back({w, u, v});
    }

    T getMstWeight()
    {
        // 排序边，按照边的权重升序排序
        std::sort(g.begin(), g.end());

        T mstWeight = 0;
        for (auto &[w, u, v] : g)
        {
            if (!dsu.iscon(u, v))
            {
                dsu.merge(u, v);
                mstWeight += w;
                --comp;
            }
        }
        if (comp != 1)
        {
            return -1;
        }
        return mstWeight;
    }
};
```

在稠密图中,边权 $m$ 达到了 $O(n^2)$ 级别，我们就考虑使用堆优化的 $prim$ 算法，$prim$ 算法类似于 $dijkstra$ 算法，我们不断从堆中取出取出当前 $dis$ 最小的点更新答案，并遍历这个点的出边更新其他点的 $dis$,朴素算法是 $O(n^2+m)$ 的，我们用 $pbds$ 中的配对堆做 $O(1)$ 修改 $key$ 值，复杂度 $O(nlogn+m)$ 需要注意的是这里的 $dis[u]$ 表示 $u$ 点到已经选定的点的最小距离 $prim$ 算法的正确性显然，相当于我们从任意一点开始，不断选最小的边开枝散叶，直至结束为止.

#### 0.17.1.2 Prim

```cpp
struct PrimMST
{
    const int INF = 1e9;                                  // 定义无穷大常量
    int n;                                                // 顶点数量(下标从1开始)
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表存储图
    // 构造函数，初始化顶点数量
    PrimMST(int n) : n(n), g(n + 1) {}
    // 添加无向边
    void addEdge(int u, int v, int w){
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }
    // 执行Prim算法，返回最小生成树权值和，无法构成返回-1
    int compute(){
        int Sum = 0;
        int Num = 0;
        // 定义配对堆类型，存储(距离，顶点)对，使用小根堆
        using PairingHeap = __gnu_pbds::priority_queue<
            std::pair<int, int>,
            std::greater<std::pair<int, int>>,
            __gnu_pbds::pairing_heap_tag>;

        PairingHeap pq;
        std::vector<PairingHeap::point_iterator> handlers(n + 1);
        // 初始化优先队列，所有节点距离设为INF，起点距离为0
        handlers[1] = pq.push({0, 1});
        for (int i = 2; i <= n; ++i){
            handlers[i] = pq.push({INF, i});
        }
        while (!pq.empty()){
            auto [dist, u] = pq.top();
            pq.pop();
            if (dist == INF){
                return -1; // 存在不连通节点
            }
            Sum += dist;
            Num++;
            handlers[u] = nullptr; // 标记已处理
            // 遍历邻接节点
            for (const auto &[v, w] : g[u]){
                if (handlers[v] != nullptr && w < handlers[v]->first) {
                    pq.modify(handlers[v], {w, v});
                }
            }
        }
        return (Num == n) ? Sum : -1;
    }
};
```

我们还有第三种 $MST$ 算法 $boruvka$，适用于其他特殊情况，在特殊条件下也十分好用
其核心思想是减少联通块的个数，我们每次遍历所有边 $(u,v,w)$ 如果 $u$ 和 $v$ 不在同一联通块内, 就用 $w$ 更新两个联通块之间的最小边, 然后每次我们将这些边加入答案，并合并连通块。不难发现每次连通块个数减少一半, 复杂度 $O(mlogn)$
常规题目我们一般不考虑此算法，但在某些题目中, 两点间的边权由特定关系决定，如果我们能快速得到最小边，就会考虑采用 $boruvka$ 算法了

#### 0.17.1.3 Boruvka

```cpp
template <class T>
struct BoruvkaMST{
    int n;                                          // 顶点数量（下标从1开始）
    int comp;                                       // 当前连通块数量
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表存储图
    DSU dsu;                                        // 并查集结构
    // 构造函数，初始化顶点数量和并查集
    BoruvkaMST(int n) : n(n), comp(n), dsu(n + 1), g(n + 1) {}
    void addEdge(int u, int v, int w){// 添加无向边
        g[u].emplace_back(v, w);g[v].emplace_back(u, w);
    }
    // 执行Borůvka算法，返回最小生成树权值和，无法构成返回-1
    T compute(){
        constexpr T inf = std::numeric_limits<T>::max() / 2;
        T totalWeight = 0;
        bool hasChanges = true;
        while (hasChanges && comp > 1){
            hasChanges = false;
            std::vector<T> min(n + 1, inf);   // 当前连通块的最小边权值
            std::vector<int> match(n + 1, 0); // 最小边对应的相邻连通块
            // 第一步：寻找每个连通块的最小出边
            for (int u = 1; u <= n; ++u){
                int rootU = dsu.root(u);
                for (const auto &[v, w] : g[u]){
                    int rootV = dsu.root(v);
                    if (rootU != rootV && w < min[rootU]){
                        min[rootU] = w;
                        match[rootU] = rootV;
                    }
                }
            }
            // 第二步：合并连通块
            for (int u = 1; u <= n; ++u) {
                if (match[u] != 0){
                    int rootU = dsu.root(u);
                    int rootV = dsu.root(match[u]);
                    if (rootU != rootV){
                        dsu.merge(rootU, rootV);
                        totalWeight += min[u];
                        --comp;
                        hasChanges = true;
                    }
                }
            }
        }
        return comp == 1 ? totalWeight : -1;
    }
};
```

### 0.17.2 kruskal 重构树

用于瓶颈路相关问题，可以在 $logn$ 时间内求出图上两点之间最大最小边权问题。但重构树并不一定只能维护边权, 例如，我们可以把边出现的时间当作边权维护，从而达到维护连通性的效果，我们适当的把其他元素转化为边权能起到很好的效果.

```cpp
class KruskalTree
{
private:
    DSU dsu;
    std::vector<int> lc, rc, val;
    int idx;
    std::vector<std::vector<int>> fa;
    std::vector<int> dep;
    int lg;

public:
    // 构造函数，初始化并构建重构树
    KruskalTree(const std::vector<std::array<int, 3>> &g, int n)
        : lc(2 * n + 1),
          rc(2 * n + 1),
```

```
        val(2 * n + 1),
        idx(n),
        dsu(2 * n),
        dep(2 * n + 1)
{

    // 计算二进制提升的最大层数
    lg = std::__lg(2 * n + 1);
    fa.assign(2 * n + 1, std::vector<int>(lg + 1, 0));

    // 按边权从大到小排序
    auto e = g;
    std::sort(e.begin(), e.end(),
              [](const auto &a, const auto &b)
              { return a[2] < b[2]; });

    // 构建Kruskal重构树
    for (const auto &[u, v, w] : e)
    {
        if (dsu.root(u) != dsu.root(v))
        {
            ++idx;
            int rootU = dsu.root(u);
            int rootV = dsu.root(v);
            lc[idx] = rootU;
            rc[idx] = rootV;
            dsu.pre[rootU] = idx;
            dsu.pre[rootV] = idx;
            val[idx] = w;
        }
        if (idx == 2 * n - 1)
        {
            break;
        }
    }

    // DFS预处理二进制提升表
    auto dfs = [&](auto &&self, int u, int parent) -> void
    {
        fa[u][0] = parent;
        for (int i = 1; i <= lg; ++i)
        {
            fa[u][i] = fa[fa[u][i - 1]][i - 1];
        }
        if (lc[u])
        {
            dep[lc[u]] = dep[u] + 1;
            self(self, lc[u], u);
        }
        if (rc[u])
        {
            dep[rc[u]] = dep[u] + 1;
            self(self, rc[u], u);
        }
    };

    // 从根节点开始DFS
    for (int i = idx; i >= 1; --i)
    {
        if (!dep[i])
        {
            dep[i] = 1;
            dfs(dfs, i, 0);
        }
    }
}

// 查询两个节点的最近公共祖先
int getLca(int u, int v) const
{
    if (dep[u] < dep[v])
    {
        std::swap(u, v);
    }
    // 提升u到与v相同深度
    for (int i = lg; i >= 0; --i)
    {
        if (dep[fa[u][i]] >= dep[v])
        {
            u = fa[u][i];
```

```
            }
        }
        if (u == v)
        {
            return u;
        }
        // 同时提升u和v
        for (int i = lg; i >= 0; --i)
        {
            if (fa[u][i] != fa[v][i])
            {
                u = fa[u][i];
                v = fa[v][i];
            }
        }
        return fa[u][0];
    }
    int getVal(int x)
    {
        return val[x];
    }
    bool isConnected(int u, int v)
    {
        return dsu.root(u) == dsu.root(v);
    }
};
```

### 0.17.3

## 0.18 网络流

网络流是一种建好图以后，内置反悔机制的一种贪心策略，不需要我们人为去寻找贪心的反悔策略

容量: 当前边最多经过的流量记作 C(u,v)

流量: 当前边实际经过的流量记作 F(u,v)

残量: C(u,v)-F(u,v)

残量网络：把残量为 0 的边删去，剩下的图称作残量网络，对应边的边权也变为残量

增广路：与二分图中增广路定义不同，增广路指残量网络上源点到汇点的一条简单路径

割: 割是一个边的集合，一个边集是割当且仅当去掉这些边后网络不在流通，即最大流为 0

最大流最小割定理: 一个网格的最大流等于其最小割

我们根据定义发现，只要存在增广路，流量就能不断增大，因此求最大流是一个求增广路的过程, 并且这条增广路我们遵循能流满就流满的贪心策略

我们选择一条增广路，假设流量为 f, 我们正向边容量-f，反向边容量 +f，这就是最开始我们说的内置反悔机制, 也就是说我们可以通过反边流量进行反悔

由于 EK 算法不优于 dinic, 我们只介绍 dinic 算法

dinic 是多路增广 + 当前弧优化的增广路算法

我们通过 bfs 给图分层，分层后多路增广, 也就是同时累加多条增广路的流量

当前弧优化是指，流量已经流满的边无用，我们没必要对每个点每次都从头开始遍历，而是记录一个当前弧，直接从当前弧开始遍历

贪心的正确性全部基于退流反边对流量不断调整直至最优状态

### 0.18.1 拆点

注意到在流网络中，我们对于限制都是加在边上的。那么对于点的限制我们往往考虑拆点。具体的说，我们把一个点 U 拆成对应的 Inu 和 Outu ，然后连边 Inu->Outu，边的流量限制即为该点的流量限制。

### 0.18.2 最大流 Dinic

```cpp
template<class T>
struct MaxFlow{
    struct _Edge{
        int to;
        T cap;
        _Edge(int _to,T _cap):to(_to),cap(_cap){};
    };

    int n;
    std::vector<_Edge> e;
    std::vector<std::vector<int>> h;
    std::vector<int> cur,d;

    MaxFlow(){};
    MaxFlow(int _n){
        init(_n);
    }
    void init(int _n){
        n=_n;
        e.clear();
        h.assign(n,{});
        cur.resize(n);
        d.resize(n);
    }
    void addEdge(int u,int v,T c){
        h[u].push_back(e.size());
        e.emplace_back(v,c);
        h[v].push_back(e.size());
        e.emplace_back(u,0);
    }

    bool bfs(int s,int t){
        d.assign(n,0);
        std::queue<int> q;
        d[s]=1;q.push(s);
        while(!q.empty()){
            int u=q.front();q.pop();
            for(int i:h[u]){
                auto [v,c]=e[i];
                if(c>0&&d[v]==0){
                    d[v]=d[u]+1;
                    if(v==t) return true;
                    q.push(v);
                }
            }
```

```
        }
        return false;
    }

    T dfs(int u,int t,T f){
        if(u==t) return f;
        T sum=f;
        for(int &i=cur[u];i<(int)h[u].size();i++){
            int j=h[u][i];
            auto [v,c]=e[j];
            if(c>0&&d[v] == d[u]+1){
                T a = dfs(v,t,std::min(sum,c));
                e[j].cap-=a;
                e[j^1].cap+=a;
                sum-=a;
                if(sum==0) return f;
            }
        }
        if(sum==f) d[u]=0;
        return f-sum;
    }

    T Dinic(int s,int t){
        T ans=0;
        while(bfs(s,t)){
            cur.assign(n,0);
            ans+=dfs(s,t,std::numeric_limits<T>::max());
        }
        return ans;
    }

    std::vector<bool> minCut(){
        std::vector<bool> c(n);
        for(int i=1;i<n;i++){
            c[i]=(d[i]!=0);
        }
        return c;
    }

    struct Edge {
        int from,to;
        T cap,flow;
    };
    std::vector<Edge> edges() {
        std::vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

### 0.18.3 最小费用最大流

```
template<class T>
struct MinCostFlow {
    int n;

    struct Edge {
        int to;
        T cap, cost;

        Edge(int to, T cap, T cost) : to(to), cap(cap), cost(cost) {}
    };

    std::vector<Edge> e;
    std::vector<std::vector<int>> h;
    std::vector<T> c, dis;
    std::vector<int> pre;

    MinCostFlow() {};

    MinCostFlow(int _n) {
        init(_n);
    }

    void init(int _n) {
        n = _n;
        e.clear();
        h.assign(n, {});
    }

    void addEdge(int u, int v, T cap, T cost) {
        h[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
        h[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    bool dijkstra(int s, int t) {
        dis.assign(n, std::numeric_limits<T>::max());
        pre.assign(n, -1);
        std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int >>, std::greater<std::pair<T, int>>> pq;
        dis[s] = 0, pq.emplace(0, s);
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (dis[u] != d) {
                continue;
            }
            for (int i: h[u]) {
                auto [v, cap, cost] = e[i];
                if (cap > 0 && dis[v] > d + c[u] - c[v] + cost) {
                    dis[v] = d + c[u] - c[v] + cost;
                    pre[v] = i;
                    pq.emplace(dis[v], v);
                }
            }
        }
        return dis[t] != std::numeric_limits<T>::max();
    }

    std::pair<T, T> flow(int s, int t) {
        T flow = 0, cost = 0;
        c.assign(n, 0);
        while (dijkstra(s, t)) {
            for (int i = 0; i < n; i++) {
                c[i] += dis[i];
            }
            T aug = std::numeric_limits<int>::max();
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                aug = std::min(aug, e[pre[i]].cap);
            }
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                e[pre[i]].cap -= aug;
                e[pre[i] ^ 1].cap += aug;
            }
            flow += aug;
            cost += aug * c[t];
        }
        return std::make_pair(flow, cost);
    }
```

```
    struct _Edge {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    std::vector<_Edge> edges() {
        std::vector<_Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            _Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }

};
```

## 0.19 连通性

### 0.19.1 有向图连通性

我们只要研究强联通分量以及其应用，一个强连通分量内的点两两可达，强联通具有传递性。

#### 0.19.1.1 强连通分量

```
class SCC
{
    const std::vector<std::vector<int>> &e; // 图的邻接表引用（输入图）
    std::vector<int> stk;                   // Tarjan算法使用的栈
    int tt = 0;                             // 栈顶指针
    int idx = 0;                            // 时间戳计数器

    // Tarjan算法的DFS核心实现
    void dfs(int x)
    {
        dfn[x] = low[x] = ++idx; // 初始化当前节点的dfn和low值
        stk[++tt] = x;           // 节点入栈

        // 遍历所有邻接节点
        for (int y : e[x])
        {
            if (!dfn[y]) // 未访问过的节点
            {
                dfs(y);                             // 递归搜索
                low[x] = std::min(low[x], low[y]); // 更新low值
            }
            else if (!bel[y]) // 已访问但未确定SCC的节点（在栈中）
            {
                low[x] = std::min(low[x], dfn[y]); // 更新low值
            }
        }

        // 发现SCC的根节点
        if (dfn[x] == low[x])
        {
            cntBlock += 1; // SCC计数器增加
            int y;
            do
            {
                y = stk[tt--];             // 弹出栈顶节点
                p[cntBlock].push_back(y); // 将节点加入当前SCC
                bel[y] = cntBlock;         // 标记节点所属SCC编号
            } while (y != x); // 直到处理到当前根节点
        }
    }

public:
    // 原始图的访问信息
    std::vector<int> dfn; // DFS访问序号（时间戳）
    std::vector<int> low; // 能回溯到的最早节点的时间戳
    std::vector<int> bel; // 节点所属的SCC编号

    // SCC相关数据
    std::vector<std::vector<int>> p; // 每个SCC包含的节点列表 1-base
    std::vector<std::vector<int>> g; // 缩点后的DAG图（邻接表）
    std::vector<int> din;            // 缩点后各节点的入度
    std::vector<int> siz;            // 各SCC的大小（节点数）
    int cntBlock = 0;                // SCC总数计数器

    // 构造函数：输入图的邻接表
    SCC(const std::vector<std::vector<int>> &e)
        : e(e),          // 初始化图引用
          dfn(e.size()), // 分配dfn数组空间
          low(e.size()), // 分配low数组空间
          bel(e.size()), // 分配bel数组空间
          p(e.size())    // 分配SCC节点列表空间
    {
        int n = e.size() - 1; // 节点数（假设节点从1开始编号）
        stk.assign(n + 1, 0); // 初始化栈

        // 对每个未访问节点执行Tarjan算法
        for (int i = 1; i <= n; i++)
        {
            if (!dfn[i]) // 未访问过的节点
            {
                dfs(i); // 开始DFS
```

```
            }
        }

        // 构建缩点后的DAG图
        g.resize(cntBlock + 1);   // 调整缩点图大小
        din.resize(cntBlock + 1); // 调整入度数组大小
        siz.resize(cntBlock + 1); // 调整SCC大小数组

        // 遍历原始图构建缩点图
        for (int x = 1; x <= n; x++)
        {
            siz[bel[x]]++;     // 统计各SCC大小
            for (int y : e[x]) // 遍历邻接节点
            {
                if (bel[x] == bel[y]) // 同一SCC内部边跳过
                {
                    continue;
                }
                g[bel[x]].push_back(bel[y]); // 添加缩点后的边
                din[bel[y]]++;               // 更新目标SCC的入度
            }
        }
    }
};
// 如果belong[u]<belong[v]且u v联通 那么缩点以后边的方向是v->u
```

#### 0.19.1.2 2-Sat

$2-Sat$ 问题是给定你一些二元布尔表达式，我们要求出满足所有表达式的一个解，或者报告无解。
需要注意的是 如果一个变量恒成立 那么我们连边 $a+n->a$
相对应的，如果一个变量恒不成立, 我们连边 $a->a+n$
如何判断无解？存在某个变量 $b$ 满足 $b$ 和 $b+n$ 在同一个 $SCC$ 内
如何输出方案？对于 $b$ 和 $b+n$ 我们取拓扑序大的为真，那么在 $SCC$ 中拓扑序越大的所在 $SCC$ 编号越小，因此输出方案为 $bel[i]<bel[i+n]$,

#### 0.19.1.3 支配树

$idom[i]$ 为 $i$ 的直接支配点，在支配树上是 $i$ 的父节点，$sdom[i]$ 是半支配点，除了辅助求支配点外无实际意义。支配树上一点的所有祖先都是该点的支配点。

```
struct DominatorTree
{
    int n, r;
    std::vector<std::vector<int>> G, rG;  // 原图和反图
    std::vector<int> dfn, idfn, parent;   // dfn[u] = DFS序, idfn[i] = DFS序为i的节点, parent[u] = DFS树父节点
    std::vector<int> sdom, idom;          // 半支配点和直接支配点
    std::vector<int> fa, mn;              // 并查集结构
    std::vector<std::vector<int>> bucket; // 分组处理sdom

    DominatorTree(int n, int r) : n(n), r(r), G(n + 1), rG(n + 1) {}

    void addEdge(int u, int v)
    {
        G[u].push_back(v);
        rG[v].push_back(u); // 同时构建反图
    }

    void dfs(int u)
    {
        dfn[u] = idfn.size();
        idfn.push_back(u);
        for (int v : G[u])
        {
            if (!dfn[v])
            { // 未访问过
                parent[v] = u;
                dfs(v);
            }
        }
    }

    // 并查集：路径压缩 + 维护sdom最小值
    int find(int u)
    {
        if (fa[u] == u)
            return u;
        int root = find(fa[u]);
        if (dfn[sdom[mn[fa[u]]]] < dfn[sdom[mn[u]]])
        {
            mn[u] = mn[fa[u]]; // 更新最小值
        }
        return fa[u] = root; // 路径压缩
    }

    // 构建支配树，返回idom数组（idom[u] = u的直接支配点）
    std::vector<int> build()
    {
        // Step 1: 初始化
        dfn.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        idfn.push_back(0); // 占位，使id[1..n]有效
        dfs(r);            // 从根节点开始DFS

        // Step 2: 初始化sdom和idom
        sdom.resize(n + 1);
        idom.resize(n + 1);
        for (int i = 1; i <= n; ++i)
        {
            sdom[i] = i;
            idom[i] = 0; // 初始化为0（无效值）
        }

        // Step 3: 并查集初始化
        fa.resize(n + 1);
        mn.resize(n + 1);
        for (int i = 1; i <= n; ++i)
```

```
        {
            fa[i] = mn[i] = i;
        }

        bucket.resize(n + 1);
        // Step 4: 按DFS序逆序处理（从大到小）
        for (int i = (int)idfn.size() - 1; i >= 2; --i)
        { // 跳过根节点（i=1）
            int u = idfn[i];

            // Step 4.1: 计算sdom[u]
            for (int v : rG[u])
            { // 遍历反图（即原图的前驱）
                if (!dfn[v])
                    continue; // 跳过不可达节点
                if (dfn[v] < dfn[u])
                {
                    sdom[u] = dfn[v] < dfn[sdom[u]] ? v : sdom[u];
                }
                else
                {
                    find(v); // 路径压缩
                    sdom[u] = dfn[sdom[mn[v]]] < dfn[sdom[u]] ? sdom[mn[v]] : sdom[u];
                }
            }
            bucket[sdom[u]].push_back(u);

            // Step 4.2: 合并到父节点
            if (parent[u])
            {
                merge(parent[u], u);
            }

            // Step 4.3: 处理bucket中的节点
            for (int v : bucket[parent[u]])
            {
                find(v);
                if (sdom[mn[v]] == parent[u])
                {
                    idom[v] = parent[u];
                }
                else
                {
                    idom[v] = mn[v]; // 暂时存储，后续修正
                }
            }
            bucket[parent[u]].clear();
        }

        // Step 5: 修正idom
        for (int i = 2; i < idfn.size(); ++i)
        { // 跳过根节点（i=1）
            int u = idfn[i];
            if (idom[u] != sdom[u])
            {
                idom[u] = idom[idom[u]];
            }
        }
        // 根节点的idom设为0（表示无支配点）
        idom[r] = 0;
        return idom;
    }

private:
    // 并查集合并操作
    void merge(int u, int v)
    {
        fa[v] = u;
    }
};
```

### 0.19.2 无向图连通性

我们主要研究点双连通分量和边双连通分量，前者对应割点，后者对应割边，割边又称桥
特别的，孤立点和孤立边的两个端点都不是割点，但孤立边是割边
我们有如下结论
1 对于 $(u,v)$ 两点间任意一条简单路径，其路径上的割边就是两点间的所有必经边，所以割边也可以看做必经边
2 对于两个点双联通分量，如果二者有公共点，那么该点一定是割点，反过来讲，该点是割点当且仅当它属于两个以上点双
3 边双具有传递性，但点双不具有传递性
4 如果两个点属于一个点双，那么这两点之间所有简单路径的并集恰好完全等于该点双
对于边双，我们可以通过缩点进一步刻画其连通性，但是点双的形态不适用于缩点，我们用广义圆方树刻画

**0.19.2.1 点双连通分量** 在模板中我们有一个割度数组 $Cutdeg$，割度表示删去该点后增加的联通分量数。割度不为 0 的点均为割点。

```
// 用于计算图的顶点双连通分量和割点（关节点）的类
class VertexBC
{
    // 图的邻接表引用（const引用避免拷贝）
    const std::vector<std::vector<int>> &adj;
    // DFS时间戳计数器，用于记录访问顺序
    int cur = 0;

    // DFS遍历函数，用于发现割点
    // x: 当前遍历的顶点
    // root: 当前连通分量的根节点
    void dfs(int x, int root)
    {
        // 初始化当前顶点的发现时间和low值
        dfn[x] = low[x] = cur++;

        // 记录在DFS树中的子节点数量
        int sonNum = 0;

        // 遍历所有邻接顶点
        for (int y : adj[x])
        {
            if (dfn[y] == -1) // 如果邻接顶点未被访问过
            {
                sonNum++;     // 增加子节点计数
                dfs(y, root); // 递归访问
                // 回溯时更新low值
                low[x] = std::min(low[x], low[y]);

                // 割点判定条件：
                // 1. 子节点的low值 >= 当前节点的dfn值
                // 2. 当前节点不是根节点
                if (low[y] >= dfn[x] && x != root)
                {
                    cutDeg[x] += 1; // 增加当前节点的割点度数
                }
            }
            else // 如果邻接顶点已被访问过（可能是回边）
            {
                // 通过回边更新low值
                low[x] = std::min(low[x], dfn[y]);
            }
        }

        // 特殊处理根节点的情况
        if (x == root)
        {
            // 根节点的割点度数 = 子树数量 - 1
            cutDeg[x] = std::max(sonNum - 1, 0);
        }
    }

public:
    // 存储每个顶点的发现时间（DFS序）
    std::vector<int> dfn;
    // 存储每个顶点的low值（能回溯到的最早祖先）
    std::vector<int> low;
    // 存储每个顶点作为割点时的度数（删除后会增加的连通分量数）
    std::vector<int> cutDeg;
    // 记录图的连通分量数量
    int componentNum = 0;
```

```
    // 构造函数，接收邻接表并计算割点信息
    VertexBC(const std::vector<std::vector<int>> &adj)
        : adj(adj),            // 初始化邻接表引用
          dfn(adj.size(), -1), // 初始化dfn数组（-1表示未访问）
          low(adj.size()),     // 初始化low数组
          cutDeg(adj.size())   // 初始化割点度数数组
    {
        int n = adj.size();

        // 遍历所有顶点（从1开始，假设顶点编号为1~n-1）
        for (int i = 1; i < n; i++)
        {
            if (dfn[i] == -1) // 遇到未访问的顶点
            {
                componentNum++; // 增加连通分量计数
                dfs(i, i);      // 从该顶点开始DFS
            }
        }
    }
};
```

广义圆方树有十分多好的性质，我们单独讨论
1 广义圆方树上没有两个邻点形状相同
2 两个圆点在圆方树上的路径，以及经过的所有方点相邻圆点的集合，等于原图中两点间所有简单路径的并集
3 根据 2 的结论，如果我们圆点赋值 $-1$，方点赋值点双大小, 那么路径权值和等于简单路径并集大小 $-2$

#### 0.19.2.2 广义圆方树

[1 , n] 为（圆点）原点，[n+1, cnt] 为方点，方点所有邻点即为该点双的点集。
额外维护了每个边属于哪个点双联通分量，在每个点双中遍历时维护新的子图，避免遍历一个点所有邻边可能导致的 O(n^2)

```
class ExRSTree
{
    // 使用 pair 存储 {目标节点 v, 边的唯一编号 id}
    const std::vector<std::vector<std::pair<int, int>>> &adj;
    int cur = 0;                                // DFS时间戳计数器

    std::vector<int> stk;                       // 用于维护点的栈
    int tt = 0;                                 // 点栈顶指针

    std::vector<int> edge_stk;                  // 修改2: 用于维护边的栈
    int edge_tt = 0;                            // 边栈顶指针

    // 在圆方树中添加边的辅助函数
    void add(int u, int v)
    {
        tree[u].push_back(v);
        tree[v].push_back(u);
    }

    // Tarjan算法实现，用于构建圆方树及边所属关系
    // x: 当前遍历的顶点
    // root: 当前连通分量的根节点
    // fa_edge: 父边的编号，防止通过无向边走回头路
    void dfs(int x, int root, int fa_edge = -1)
    {
        dfn[x] = low[x] = cur++; // 初始化发现时间和low值
        stk[++tt] = x;           // 将当前节点压栈

        int sonNum = 0; // DFS树中子节点计数

        for (auto edge : adj[x]) // 遍历所有邻接边
        {
            int y = edge.first;
            int id = edge.second;

            if (dfn[y] == -1) // 未访问过的节点
            {
                sonNum += 1;
                edge_stk[++edge_tt] = id; // 【边入栈】将树边压栈

                dfs(y, root, id);
                low[x] = std::min(low[x], low[y]); // 更新low值

                // 发现割点条件（即找到了一个新的点双连通分量）
                if (low[y] >= dfn[x])
                {
                    ++cnt; // 新增一个方点

                    // 1. 弹出点栈中元素，构建圆方树的点关系
                    while (stk[tt] != y)
                    {
                        Size[cnt]++;          // 更新点双大小
                        add(cnt, stk[tt--]); // 将圆点连接到方点
                    }
                    // 处理剩余部分
                    add(cnt, stk[tt--]); // 连接y到方点
                    add(cnt, x);         // 连接当前割点到方点
                    Size[cnt] += 2;      // 更新点双大小（加上x和y）

                    // 2. 弹出边栈中元素，记录边所属的点双（方点编号）
                    while (true)
                    {
                        int cur_edge = edge_stk[edge_tt--];
                        edge_bel[cur_edge] = cnt; // 记录当前边属于新增的方点 cnt
                        if (cur_edge == id) break; // 直到弹出引发当前点双的树边为止
                    }
                }
            }
```

```
            else if (id != fa_edge && dfn[y] < dfn[x])
            {
                // 遇到返祖边，同样属于某个点双，需要将其压入边栈
                // dfn[y] < dfn[x] 保证只压入指向祖先的返祖边，避免重复压栈
                edge_stk[++edge_tt] = id;
                low[x] = std::min(low[x], dfn[y]); // 更新low值
            }
        }
    }

    // 第二次DFS，计算子树大小和所属根节点
    void dfs1(int u, int fa, int r)
    {
        if (u <= n) // 如果是圆点（原始节点）
        {
            siz[u] = 1; // 初始化大小为1
        }

        bel[u] = r; // 记录所属根节点

        for (auto v : tree[u]) // 遍历圆方树中的邻接节点
        {
            if (v != fa) // 避免回溯父节点
            {
                dfs1(v, u, r);
                siz[u] += siz[v]; // 累加子树大小
            }
        }
    }

public:
    // original graph
    int n;                                  // 原始图的顶点数
    int m;                                  // 原始图的边数
    int cnt;                                // 圆方树中的节点计数器（圆点+方点）
    std::vector<int> dfn, low;              // Tarjan算法用的发现时间和low值
    std::vector<std::vector<int>> tree; // 圆方树
    int componentNum = 0;                   // 连通分量数量
    std::vector<int> root;                  // 所有圆方树的根节点
    std::vector<int> bel;                   // 节点所属的圆方树根节点
    std::vector<int> siz;                   // 子树大小
    std::vector<int> Size;                  // 点双连通分量的大小

    std::vector<int> edge_bel;              // 【新增】edge_bel[id] 表示编号为 id 的边所属的方点（点双）编号

    // 构造函数：从原始图构建圆方树
    // 参数 m 为边的总数量，用于初始化边的 belong 数组
    ExRSTree(const std::vector<std::vector<std::pair<int, int>>> &adj, int m)
        : adj(adj),
          n(adj.size() - 1), // 假设顶点编号从1开始
          m(m),
          dfn(n + 1, -1),    // 初始化dfn数组
          low(n + 1),        // 初始化low数组
          tree(2 * n + 1),   // 圆方树最多有2n个节点（n圆点+n方点）
          bel(2 * n + 1),    // 所属根节点数组
          siz(2 * n + 1),    // 子树大小数组
          edge_bel(m+1)        // 边所属点双数组，假设边编号为 0 ~ m-1
    {
        Size.assign(2 * n + 1, 0); // 初始化点双大小数组
        stk.assign(n + 1, 0);      // 初始化Tarjan点栈
        edge_stk.assign(m + 1, 0); // 初始化Tarjan边栈
        cnt = n;                   // 方点从n+1开始编号

        // 遍历所有顶点构建圆方树
        for (int i = 1; i <= n; i++)
        {
            if (dfn[i] == -1) // 未访问过的顶点
            {
                root.push_back(i); // 记录根节点
                componentNum += 1; // 增加连通分量计数
                dfs(i, i);         // 第一次DFS构建圆方树
                dfs1(i, i, i);     // 第二次DFS计算子树信息
            }
        }
    }
};
```

#### 0.19.2.3 边双连通分量

```
class EdgeBC
{
    // 输入的无向图的邻接表（引用，避免拷贝）
    const std::vector<std::vector<int>> &e;
    // Tarjan算法用的栈
    std::vector<int> stk; // stack
    int tt = 0;           // 栈顶指针
    int cur = 0;          // 全局时间戳，用于dfn和low的标记
    // Tarjan算法求边双连通分量（EBCC）
    void dfs(int x, int fa){
        // 初始化当前节点的dfn和low值为当前时间戳
        dfn[x] = low[x] = ++cur;
        // 将当前节点压入栈
        stk[++tt] = x;
        // 遍历所有邻接节点
        for (int y : e[x]){
            // 跳过父节点（处理无向图的重复边）
            if (y == fa){
                fa = -fa; // 特殊处理，确保每条边只处理一次
                continue;
            }
            // 如果邻接节点y未被访问过
            if (!dfn[y]) {
                dfs(y, x); // 递归访问y
                low[x] = std::min(low[x], low[y]); // 更新low值
            }else{
                // y已被访问，更新low值
                low[x] = std::min(low[x], dfn[y]);
            }
        }
        // 如果dfn[x] == low[x]，说明x是一个边双连通分量的根
        if (dfn[x] == low[x]){
            int y;
            cntBlock += 1; // 边双连通分量计数
            // 弹出栈中元素，直到x被弹出，这些元素构成一个边双连通分量
            do{
                y = stk[tt--];
                ebcc[cntBlock].push_back(y); // 将y加入当前边双连通分量
                bel[y] = cntBlock;           // 记录y所属的边双连通分量编号
            } while (y != x);
        }
    }
public:
    // 原始图的属性
    std::vector<int> dfn;  // 访问时间戳（DFS序）
    std::vector<int> low;   // 通过回边能到达的最小dfn值
    // low[x] 表示从 x 节点及其子树出发，经过任意条树边和至多一条返祖边所能到达的最小时间戳（dfn）。
    std::vector<int> bel;   // 每个节点所属的边双连通分量编号
    std::vector<std::vector<int>> ebcc; // 存储每个边双连通分量的节点列表
    // 缩点后的图（边双连通分量构成的树/森林）
    std::vector<std::vector<int>> g; // 缩点后的邻接表
    std::vector<int> siz;           // 每个边双连通分量的大小
    std::vector<int> val;           // 每个边双连通分量的权值（可选）
    int cntBlock = 0;               // 边双连通分量的总数
    // 构造函数：输入无向图的邻接表e（节点编号从1开始）
    EdgeBC(const std::vector<std::vector<int>> &e) :
        e(e),                       // 初始化图的引用
        dfn(e.size()),              // 初始化dfn数组
        low(e.size()),              // 初始化low数组
        bel(e.size()),              // 初始化bel数组
        stk(e.size()),              // 初始化栈
        ebcc(e.size())              // 初始化ebcc数组
    {
        int n = e.size() - 1; // 节点数（假设节点编号为1~n）
        // 遍历所有节点，对未访问的节点调用Tarjan算法
        for (int i = 1; i <= n; i++){
            if (!dfn[i]){
                dfs(i, -1); // 从i开始DFS，初始父节点为-1
            }
        }
        // 调整ebcc的大小为实际边双连通分量的数量
        ebcc.resize(cntBlock + 1);
        // 初始化缩点后的图
        g.resize(cntBlock + 1);
        siz.resize(cntBlock + 1);
        // val.resize(cntBlock + 1); // 如果需要权值，可以取消注释
        // 遍历所有节点，构建缩点后的图
```

```
        for (int x = 1; x <= n; x++){
            siz[bel[x]]++; // 统计每个边双连通分量的大小
            // val[bel[x]] += a[x]; // 如果需要权值，可以累加

            // 遍历x的邻接节点y
            for (int y : e[x]){
                // 如果x和y属于不同的边双连通分量，则在缩点后的图中添加边
                if (bel[x] != bel[y]){
                    g[bel[x]].push_back(bel[y]);
                }
            }
        }
    }
};
```

#### 0.19.2.4 构造边双连通图

即给你一个无向图，加最少的边使其成为边双连通图。首先我们 $EBCC$ 缩点，这样一定会形成一颗森林，

我们假如说这是一棵树的话，记叶子结点数量为 $leaf$，那么答案为 $(leaf+1)/2$，构造方法是，我们把 $dfs$ 得到的叶子结点按遍历顺序放到数组中，然后不断首尾匹配即可。

那么如果是森林，我们考虑先把它连成一棵树使得该树的叶子数量最少，在不考虑孤立点的情况下，每棵树至少会有两个叶子，那么我们把这两个叶子找出来，把这些森林按顺序首尾链接即可，注意需要特判孤立点的情况。

假如有 $C$ 个联通块，那么这里消耗边数

## 0.20 欧拉图

欧拉路径：经过每条边恰好一次的路径
欧拉回路：经过每条边恰好一次并回到起点的路径
欧拉图：有欧拉回路的图被称作欧拉图
半欧拉图：没有欧拉回路但有欧拉路径的图被称作半欧拉图
欧拉图判断条件：
对于连通图 $G$，如下三个性质等价：
1：$G$ 是欧拉图。
2：对于无向图，$G$ 中所有顶点的度数都是偶数；对于有向图，每个点入度等于出度。
3：$G$ 可以被分解成若干条不共边的回路。
半欧拉图判断条件：
对于有向图：有一个点 $in = out + 1$, 有一个点 $out = in + 1$, 其余点 $in = out$
对于无向图，有两个点 $deg$ 为奇数。
显然对于有向图和无向图，两个特殊点都是欧拉通路的两个端点。

### 0.20.1 无向图

注意代码求出的是字典序最小的欧拉路。

```
class UndirectedEulerRoad
{
public:
    int n, m;
    // pair.first: 目标节点 v, pair.second: 边的全局唯一 ID
    std::vector<std::vector<std::pair<int, int>>> g;
    std::vector<int> cnt, deg, road;
    std::vector<bool> vis_edge; // 记录某条边是否被走过（替代繁琐的反向边删除）

    UndirectedEulerRoad(int _n) : n(_n), m(0), g(_n + 1)
    {
        cnt.assign(n + 1, 0);
        deg.assign(n + 1, 0);
    }

    void addEdge(int u, int v)
    {
        // 赋予这条边一个唯一的编号 m
        g[u].push_back({v, m});
        g[v].push_back({u, m});
        deg[u]++;
        deg[v]++;
        m++; // 总边数加 1
    }

    void hierholzer(int x)
    {
        while (cnt[x] < g[x].size())
        {
            auto [v, id] = g[x][cnt[x]++];
            // 如果这条边（包含它的反向边）已经被走过，直接跳过
            if (vis_edge[id]) continue;

            vis_edge[id] = true; // 标记边已走
            hierholzer(v);
        }
        road.push_back(x);
    }

    int work()
    {
        vis_edge.assign(m, false);

        // 保证字典序，pair 默认先比较 first(即节点 v)，这正是我们需要的
        for (int i = 1; i <= n; ++i)
        {
            std::sort(begin(g[i]), end(g[i]));
        }

        int start = -1, first_odd = -1;
        int odd_cnt = 0;

        for (int i = 1; i <= n; ++i)
        {
```

```
            // 备选起点：如果有度数，可以作为回路的起点（保证字典序找编号最小的）
            if (deg[i] > 0 && start == -1) start = i;

            if (deg[i] % 2 != 0) // 无向图只需要看度数的奇偶性
            {
                odd_cnt++;
                if (first_odd == -1) first_odd = i; // 记录编号最小的奇数点
            }
        }

        // 图全空
        if (start == -1) return 1;

        // 无向图欧拉路径只有两种情况：0 个奇点(回路）或 2 个奇点(通路）
        if (odd_cnt != 0 && odd_cnt != 2) return 0;

        // 如果是通路，起点必须是编号最小的奇数点（为了字典序）
        if (odd_cnt == 2) start = first_odd;

        hierholzer(start);

        // 连通性校验：防止图中有未走到的孤立边或环
        if (road.size() != m + 1) return 0;

        return (odd_cnt == 2 ? 1 : 2); // 1表示通路，2表示回路
    }

    auto getRoad()
    {
        std::reverse(begin(road), end(road));
        return road;
    }
};
```

### 0.20.2 有向图

注意代码求出的是字典序最小的欧拉路, 如果让走的路都权值字典序最小稍微改改就行。

```
class DirectedEulerRoad
{
public:
    int n, m;                           // m 用于最后判断是否连通
    std::vector<std::vector<int>> g;  // 有向图不需要 exist 标记，直接存目标点即可
    std::vector<int> cnt, in, out, road;
    DirectedEulerRoad(int _n) : n(_n), m(0), g(_n + 1)
    {
        cnt.assign(n + 1, 0);
        in.assign(n + 1, 0), out.assign(n + 1, 0);
    }
    void addEdge(int u, int v)
    {
        g[u].push_back(v);
        in[v]++;
        out[u]++;
        m++; // 统计总边数
    }
    void hierholzer(int x)
    {
        while (cnt[x] < g[x].size())
        {
            int v = g[x][cnt[x]++];
            hierholzer(v);
        }
        road.push_back(x);
    }
    int work()
    {
        // 保证字典序
        for (int i = 1; i <= n; ++i)
        {
            std::sort(begin(g[i]), end(g[i]));
        }
        int start = -1;
        int tot1 = 0, tot2 = 0;
        for (int i = 1; i <= n; ++i){
            // 如果是回路，备选一个有出度的点作起点
            if (out[i] > 0 && start == -1) start = i;
            if (in[i] != out[i])
            {
                if (out[i] - in[i] == 1) {
                    start = i; // 通路必须从 out 比 in 大 1 的点出发
                    tot1++;
                } else if (in[i] - out[i] == 1) {
                    tot2++;
                } else {
                    return 0; // 度数差大于1，绝对无解
                }
            }
        }


        // 图全空
        if (start == -1) return 1;
        // 终点和起点必须成对出现（0个或1个）
        if (!((tot1 == 1 && tot2 == 1) || (tot1 == 0 && tot2 == 0))) return 0;
        hierholzer(start);
        // 连通性校验：如果走过的点数不等于边数+1，说明有未走到的孤立环
        if (road.size() != m + 1) return 0;

        return (tot1 == 1 ? 1 : 2); // 1表示通路，2表示回路
    }
    auto getRoad(){
        std::reverse(begin(road), end(road));
        return road;
    }
};
```

### 0.20.3 混合图

混合图即既有有向边又有无向边，之所以不能转化为有向图，是因为在该问题下，无向边被视作可定向的有向边
判断是否有欧拉回路：

首先给每条无向边任意定向，设 $deg[x] = dout[x] - din[x]$
若存在 $deg[x]\&1$ 或者图不联通 直接无解
否则我们建立源点 $S$ 和汇点 $T$
对于一个点 $x$, 如果 $deg[x] > 0$，那么 $S$ 向 $x$ 流量为 $deg[x]/2$ 的边，否则 $x$ 向 $T$ 连流量为 $-deg[x]/2$ 的边，对于定向的无向边 $x->y$，连边 $(x, y, 1)$，跑最大流算法
如果与 $S$ 和 $T$ 相连的所有边均满流，那么说明有解。
同时该条件等价于 $maxFlow = \sum max((din - dout)/2, 0)$，即最大流等于所有入度大于出度节点对应的入度减出度除以二的和
构造方案：根据网络流的结果调整无向边的方向。将网络流中加入的定向边流量为 $1$ 的边翻转

## 0.21 拉姆齐定理

有 $X$ 个人，任意两两之间认识或不认识，我们定义 $R(n,m)$ 为 保证“存在一个 m 人的朋友小团体（红色 $K_m$（红色边表示朋友））”或“一个 n 人的陌生人小团体（蓝色 $K_n$(蓝色边表示陌生)）”所需要的最小派对人数

朋友小团体指该团体内两两互为朋友，陌生人即该团体内两两均不认识。

$R(3,3)=6, R(4,4)=18$

## 0.22 竞赛图

任意两点之间有且仅有一条有向边的图为竞赛图，由于连边方式类似于任意两人都打一场单循环赛，得名竞赛图，特别的，我们一般认为有向边的起点是获胜者，因此每个点的出度为对应选手的得分。

得分序列：把每个点的出度拿出来升序排序后的序列。

**0.22.0.1 兰道定理** 对于任意一个单调不减的序列，该序列可以为竞赛图的得分序列当且仅当：

1 对任意的 $i \leq n$，前 $i$ 个得分和一定不小于 $i*(i-1)/2$

2 总和必须等于 $n*(n-1)/2$

若满足兰道定理，我们一定可以构造出满足条件的竞赛图。

构造方法：我们不妨设每个选手有一个需求值 $a[i]-b[i]$，其中 $a$ 是需要赢多少场，$b$ 是已经赢多少场，我们每次让需求最大的选手打赢需求最小的选手即可。

## 0.23 二分图

二分图是无向图的概念，二分图是一类特殊的无向图. 其特点是可以把点分成左右两堆，满足每堆内部的点两两不存在有边，特别的我们称为左部点和右部点

### 0.23.1 二分图的判断

一个无向图是二分图的充要条件是不存在奇环 我们有两种方法判断二分图

1 我们可以通过二分图染色来判断是否为二分图, 如果是二分图，染色过程中还帮我们分出了左部点和右部点，时间复杂度 $O(n+m)$

2 如果像线段树分治那样，需要动态判断二分图，我们可以采用扩展域并查集，对于边 $(u,v)$ 我们 $merge(u,v+n), merge(v,u+n)$, 加边 $(u,v)$ 时我们先 $check\,(u,v)$ 是否连通，如果联通则说明有奇环，为什么呢？注意我们的连边方式，想要直接连通必须经过偶数条边，当再出现一条直接联通的边时，就存在了奇环

#### 0.23.1.1 二分图染色

```
struct BiGraphColor
{
    std::vector<int> col;
    bool isBiGraph;
    int cnt1 = 0,cnt2 = 0;
    BiGraphColor(const std::vector<std::vector<int>> &e) : col(e.size(), -1), isBiGraph(true)
    {
        int n = e.size() - 1;
        std::function<void(int)> dfs = [&](int x){
            if (!isBiGraph){
                return;
            }
            for (int y : e[x]){
                if (col[y] == -1){
                    col[y] = col[x] ^ 1;
                    if (col[y] == 0){
                        cnt1++;
                    } else{
                        cnt2++;
                    }
                    dfs(y);
                }
                else if (col[y] == col[x]){
                    isBiGraph = false;
                    return;
                }
            }
        };
        for (int i = 1; i <= n; i++){
            if (col[i] == -1){
                col[i] = 0;
                dfs(i);
            }
            if (!isBiGraph) return;
        }
    }
};
```

二分图匹配问题留在图匹配中统一说明

### 0.23.2 二分图上博弈

在一张二分图上，$Alice$ 和 $Bob$ 从某个起点开始轮流移动，$Alice$ 先手，不能重复移动到同一节点，无法移动者输？问必胜策略
结论：如果起点是最大匹配的关键点，那么 $Alice$ 必胜
证明:$Alice$ 只要按匹配边不断走就可以，我们可以证明 $Bob$ 无法在过程中走到未盖点，假如 $Bob$ 走到未盖点，那我们翻转 $Alice$ 的起点出发的路径，匹配数不变，但此时起点不再属于匹配点，与起点是关键点矛盾。因此 $Bob$ 只能跟随 $Alice$ 不断走交错路，直至无路可走
我们只需找到关键点即可，在二分图匹配中我们已经详细阐述了如何寻找关键点

### 0.23.3 删边二分图

给你一个无向图问删除一条边是不是二分图?
我们从维护奇环的角度来考虑，没有奇环是二分图的充要条件，我们考虑对图跑出 $dfs$ 树，然后考虑非树边在树上构成的环路长度，我们每找到一个奇环，就对这条路径上的树边进行区间加, 并标记该非树边，同理我们每找到一个偶环，也对这条路径上的树边区间加，我们用树上差分实现。十分关键的一点是，我们为什么要统计偶环呢？我们发现这种操作方式并不能找到所有的环，对于环套环的图即可发现，也就是说我们可能遗漏某些环的信息，具体的说，如果两个奇环合并又构成一个偶环，我们只会统计到一个奇环和一个偶环，我们知道只有两个奇环共用的边才是合法答案，而我们知道奇环 + 偶环 = 奇环，如果一条边既被奇环覆盖，又被偶环覆盖，说明他可以衍生出新的奇环，这样该边就不合法了，因为我们删去这条边需要删掉所有的奇环
所以我们得到统计方法:
对于没有奇环的图，每条边都是合法边
对于存在奇环的图，对于树边，如果该边被奇环覆盖的次数等于统计的奇环个数，且该边不被偶环覆盖，那么是合法边。对于非树边，当且仅当只有一个奇环，且该树边构成这个奇环时才合法

### 0.23.4 情况一：`K = 0`

图本身就是二分图：所有边合法

---

### 0.23.5 情况二：`K > 0`

对于**树边** `e`：$odd[e] = K \quad \wedge \quad even[e] = 0$ 才合法。
含义：

- `odd[e]=K`：所有已发现的奇环都经过它
- `even[e]=0`：不存在通过偶环组合出“绕开它的新奇环”的情况

---

对于**非树边** `e`：$K = 1 \quad \wedge \quad e$ 属于这个唯一奇环 才合法。

```cpp
#include <algorithm>
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int inf = 1e18;

void init(){}
#define MultiTest   0
void solve(){
    int n,m;cin>>n>>m;
    vector<vector<pair<int,int>>> e(n+1);
    vector<pair<int,int>> edge;
    for(int i = 1;i<=m;i++){
        int u,v;cin>>u>>v;
        edge.push_back({u,v});
        e[u].emplace_back(v,i);
        e[v].emplace_back(u,i);
    }
    vector<int> vis(n+1),dfn(n+1),dis(n+1),fa(n+1);
    vector<int> even(n+1),odd(n+1);
    int K = 0,idx = 0,oddEdge = -1;
    auto dfs = [&](auto dfs,int u,int pe)->void{
        dfn[u] = ++idx;
        vis[u] = 1;

        // 发现返祖边
        for(auto [v,id]:e[u]){
            if(id == pe) continue;
            if(vis[v] and dfn[v]<dfn[u]){
                if((dis[u]-dis[v])&1){
                    even[u]++;
                    even[v]--;
                }else{
```

```
                    K++;
                    odd[u]++;
                    odd[v]--;
                    oddEdge = id;
                }
            }
        }
        for(auto [v,id]:e[u]){
            if(id == pe || vis[v]) continue;
            fa[v] = id;
            dis[v] = dis[u]+1;
            dfs(dfs,v,id);
            even[u] += even[v];
            odd[u] += odd[v];
        }
    };
    for(int i = 1;i<=n;i++){
        if(!vis[i]) dfs(dfs,i,0);
    }
    if(K == 0){
        cout<<m<<"\n";
        for(int i = 1;i<=m;i++){
            cout<<i<<" ";
        }
        cout<<"\n";
        return ;
    }
    vector<int> ans;
    if(K == 1) ans.push_back(oddEdge);
    for(int i = 1;i<=n;i++){
        if(fa[i] != 0 and odd[i] == K and even[i] == 0){
            ans.push_back(fa[i]);
        }
    }
    cout<<ans.size()<<"\n";
    sort(ans.begin(),ans.end());
    for(int x:ans){
        cout<<x<< " ";
    }
    cout<<"\n";
}

signed main(){
    std::cin.tie(nullptr)->sync_with_stdio(false);
    init();
    int T = 1;
    if(MultiTest)
        std::cin>>T;
    while(T--) solve();
    return 0;
}
```

## 0.24 图匹配

### 0.24.1 基础概念

**匹配：**

对于一张图 $G=(V,E)$, 我们有一个边集 $E'$ 是 $E$ 的子集，且满足 $E'$ 中的边两两之间无公共点, 那么 $E'$ 称作该图的一个匹配。在匹配中的边称作匹配边，否则称作非匹配边，同理我们称点为已盖点和未盖点

**增广路:**

两端都是未盖点的简单路径，且路径上的边满足匹配边和非匹配边交替的路径称作增广路。用边集表示类似于 $01010$, 用点集表示即 $0111110$ 增广路的意义在于你翻转所有匹配边和非匹配边，得到的匹配边加一。

**最大匹配：**

在图的所有匹配中，如果一个匹配 $E'$ 包含的元素数量是最多的，那么我们称作最大匹配，特别的，一个图不一定只有一个最大匹配

**匹配关键点**

在一个图的所有最大匹配中都存在的点称作匹配的关键点

**增广路定理：**

一个匹配是图的最大匹配当且仅当当前残图上的未盖点和非匹配边不存在增广路

**完美匹配：**

如果一个匹配 $E'$ 中的边已经涵盖了图上原有的所有点，那么该匹配称作完美匹配

**最大权匹配：**

当我们的边存在边权时，匹配中边权和最大的匹配称作最大权匹配

**最大权完美匹配**

即边权和最大的完美匹配

**最小点覆盖**

对一个图来说，我们用最少的点覆盖所有的边，此时点的数量称作最小点覆盖

**最小边覆盖**

对一个图来说，我们用最少的边覆盖所有的点，此时边的数量称作最小边覆盖

**最大独立集**

对一个图来说，我们选出最多的点，满足这些点之间没有边直接相连，此时点的数量称作最大独立集

### 0.24.2 二分图上的匹配

匹配在二分图中将有很好的特殊性质，因此我们把二分图和一般图分开讨论，我们用 $G(V1, V2, E)$ 表示二分图,$V1$ 表示左部点的集合，$V2$ 表示右部点的集合
特别的如果题目没有帮我们区分左部点和右部点, 我们可以二分图染色得到

**0.24.2.1 二分图最大匹配** 匈牙利算法：根据增广路的概念和增广路定理，我们只需要朴素的不断寻找增广路即可，直至我们找不到增广路由此诞生了匈牙利算法，匈牙利算法的原理是对每一个左部的未盖点，我去右部同样寻找一个未盖点与其配对形成增广路，这样匹配数 $+1$，但算法过程略有不同，对于左部点 $V1$，假设找到了右部点 $V2$ 但 $V2$ 已经匹配，我们尝试递归 $V2$ 已匹配的左部点 $V3$ 使其更换匹配点，如果 $V3$ 能找到新匹配点 $V4$, 那么 $V3$ 和 $V4$ 配对，$V1$ 和 $V2$ 配对。对每一个左部点都进行此操作，最终成功配对数量即最大匹配, 复杂度 $O(n * E + m)$ 其中 $n$ 为左部点数量，$E$ 为边的数量，$m$ 为右部点数量，不难发现我们可以使点数少的点为左部来优化时间
在题目中，如果明确了左部点右部点，不能进行二分图染色!
第一个是手动二分图染色版本

```
struct Hungary
{
    int n;
    const std::vector<std::vector<int>> &g;
    std::vector<int> left, right, col;
    bool isBigraph;
    int maxMatch;
    std::vector<int> vis, match; // match记每个右部点匹配的左部点是谁
    int cnt = 0;
    void dfs1(int u) // 二分图染色
    {
        for (auto v : g[u])
        {
            if (col[v] == -1)
            {
                col[v] = col[u] ^ 1;
                if (col[v] == 1)
                {
                    right.push_back(v);
                }
                else
                {
                    left.push_back(v);
                }
                dfs1(v);
            }
            else if (col[u] == col[v])
            {
                isBigraph = false;
                return;
            }
        }
    }
    bool dfs2(int u, int num) // 找增广路
    {
        for (auto v : g[u])
        {
            if (vis[v] == num)
            {
                continue;
            }
            vis[v] = num;
            if (!match[v] or dfs2(match[v], num))
            {
                match[v] = u;
                return true;
            }
        }
        return false;
    }
    Hungary(const std::vector<std::vector<int>> &g) : g(g), n(g.size() - 1), vis(n + 1), match(n + 1)
    {
        isBigraph = 1;
        maxMatch = 0;
        col.assign(n + 1, -1);
        for (int i = 1; i <= n; ++i)
        {
            if (col[i] == -1)
            {
                col[i] = 0;
                left.push_back(i);
                dfs1(i);
            }
```

```
        }
        if (isBigraph)
        {
            if (left.size() > right.size())
            {
                std::swap(left, right);
            }
            for (auto i : left)
            {
                ++cnt; // 时间戳优化
                if (dfs2(i, cnt))
                {
                    ++maxMatch;
                }
            }
        }
    }
};
```

第二个是传入图和左部点集版本

```
struct Hungary
{
    int n;
    const std::vector<std::vector<int>> &g;
    std::vector<int> left;
    int maxMatch;
    std::vector<int> vis, match; // match记每个右部点匹配的左部点是谁
    int cnt = 0;
    bool dfs2(int u, int num) // 找增广路
    {
        for (auto v : g[u])
        {
            if (vis[v] == num)
            {
                continue;
            }
            vis[v] = num;
            if (!match[v] or dfs2(match[v], num))
            {
                match[v] = u;
                return true;
            }
        }
        return false;
    }
    Hungary(const std::vector<std::vector<int>> &g, std::vector<int> L) : g(g), n(g.size() - 1), vis(n + 1), match(n + 1)
    {
        left = L;
        maxMatch = 0;
        for (auto i : left)
        {
            ++cnt; // 时间戳优化
            if (dfs2(i, cnt))
            {
                ++maxMatch;
            }
        }
    }
};
```

最大流做法，我们建立超级源点 $S$ 和超级汇点 $T$，$S$ 向每一个左部点连流量为 $1$ 的边, 每一个右部点向 $T$ 连流量为 $1$ 的边，二分图原有边流量置为 $1$，从 $S$ 到 $T$ 的最大流即最大匹配，正确性显然

重边并不会影响算法的正确性，因为源点汇点的流量卡死了.

#### 0.24.2.2 匹配的关键点

首先我们有朴素算法，即去掉每个点后再次跑最大匹配，看匹配的大小是否改变，但是我们有更优秀的方法

我们给出一个引理:

一个匹配点 $P$ 不是关键点，当且仅当存在一条以 $P$ 为端点，以匹配边出发的交替链，使得终点为某个未盖点

因此我们得出做法: 先跑出最大匹配，然后我们从左边的每一个未盖点出发，给路径上的点打标记，然后左边未被标记的匹配点就是关键点，同理我们可以得到右边的关键点。复杂度 $O(V+E)$

我们也可以用最大流跑出流网络然后再使用同样的方法得到关键点。

```
class HungaryCriticalPoints
{
private:
    int n;                                  // 顶点数量
    const std::vector<std::vector<int>> &g; // 图的邻接表表示
    std::vector<int> left;                  // 左部点集合
    std::vector<int> matchl;                // 左部点的匹配点 (matchl[u] = v 表示左部点u匹配右部点v)
    std::vector<int> matchr;                // 右部点的匹配点 (matchr[v] = u 表示右部点v匹配左部点u)
    std::vector<int> pos;                   // 标记是左部点 还是右部点 左部点为1 右部点为0
public:
    /**
     * @brief 构造函数，初始化匈牙利算法关键点查找器
     * @param g 图的邻接表表示，g.size() = n+1，顶点编号从1开始
     * @param left 左部点集合
     */
    HungaryCriticalPoints(const std::vector<std::vector<int>> &g, std::vector<int> left)
        : g(g), n(g.size() - 1), left(left)
    {
        // 初始化匹配数组
        pos.resize(n + 1, 0);
        matchl.resize(n + 1, 0);
        matchr.resize(n + 1, 0);
        for (auto x : left) // 左部点标记为1
        {
            pos[x] = 1;
        }
        // 运行匈牙利算法找到最大匹配
        Hungary hungary(g, left);
        // 将匹配结果存入matchl和matchr
        for (int v = 1; v <= n; ++v)
        {
            if (hungary.match[v] != 0)
            {
                matchl[hungary.match[v]] = v;
                matchr[v] = hungary.match[v];
            }
        }
    }

    /**
     * @brief 查找所有关键点
     * @return 返回一个pair，first是左部关键点，second是右部关键点
     */
    std::pair<std::vector<int>, std::vector<int>> findCriticalPoints()
    {
        std::vector<bool> markl(n + 1, false); // 标记左部点是否被访问 未被访问是关键点
        std::vector<bool> markr(n + 1, false); // 标记右部点是否被访问

        // 从所有未匹配点出发进行BFS 左右两边分别做
        std::queue<int> ql, qr;
        for (int i = 1; i <= n; ++i){
            if (pos[i] and matchl[i] == 0) {
                ql.push(i);
                markl[i] = 1;
            }
            else if (!pos[i] and matchr[i] == 0) // 未盖点
            {
                qr.push(i);
                markr[i] = 1;
            }
        }
        // BFS标记交替路径上的点
        while (!ql.empty()){
            int u = ql.front();
            ql.pop();
            for (auto v : g[u]){
                v = matchr[v];
                if (!markl[v]){
                    ql.push(v);
                    markl[v] = 1;
                }
```

```
            }
        }
        while (!qr.empty()){
            int u = qr.front();
            qr.pop();
            for (auto v : g[u]){
                v = matchl[v];
                if (!markr[v]){
                    qr.push(v);
                    markr[v] = 1;
                }
            }
        }
        // 收集关键点
        std::vector<int> leftCritical, rightCritical;
        for (int i = 1; i <= n; ++i)
        {
            if (pos[i] and !markl[i]){
                leftCritical.push_back(i);
            }
            else if (!pos[i] and !markr[i]){
                rightCritical.push_back(i);
            }
        }
        return {leftCritical, rightCritical};
    }
};
```

二分图最大匹配的性质

最大匹配 = 最小点覆盖

最大匹配 = $n-$ 最大独立集

最大匹配 = $n-$ 最小边覆盖 - 孤立点数

$Hall$ 定理:

对于一个二分图 $G = (V1, V2, E)$, 我们设 $S$ 为 $V1$ 的一个子集，$F(S)$ 为 $S$ 的一个邻集，即 $S$ 所有相邻点的集合，总有 $|F(S)| \geq |S|$, 也就是说左部点的子集的邻集元素数量不小于子集的元素数量

$Hall$ 定理加强描述:

更强的结论是，$G$ 的最大匹配为 $|V1| - max_{S \subseteq V1}(|S| - |F(S)|)$

最小点覆盖的构造方法: 对于左部每个未匹配点，出发走交替路，给经过的所有点打标记。那么最小点覆盖的点集为: 左部所有不可达点和右部所有可达点

最大独立集的构造方法: 即最小点覆盖的补集

最小边覆盖的构造方法: 未匹配点随机选一条出边即可

#### 0.24.2.3 二分图最大多重匹配

也即我们对 $u$ 点设一个变量 $h(u)$，代表 $u$ 点不能与超过 $h(u)$ 条边相连, 一般的最大匹配中 $h(u) = 1$

我们只需要把 $S - V1$,$V2 - T$ 的流量改为对应的 $h(u)$ 跑最大流即可

#### 0.24.2.4 二分图带权最大匹配

对于最大权或者最小权最大匹配，我们改最大流为最大费用最大流或最小费用最大流

#### 0.24.2.5 二分图最大权完美匹配

我们用 $KM$ 算法在 $O(n^3)$ 的时间复杂度内解决该问题

我们默认左部右部点数量相等，否则将是最大权匹配

需要注意的是，如果题目存在负边权，那么不存在的边视作负无穷大，否则视作 $0$

首先我们为每个点赋一个权值 $h(u)$ 称作顶标, 要求对于所有的边 $(u,v,w)$，都满足 $h(u)+h(v)\geq w$

然后我们引出相等子图的概念，相等子图即只包含满足 $h(u)+h(v)=w$ 的边的子图

然后我们有定理：如果相等子图存在完美匹配，那么该匹配为最大权完美匹配

所以求解最大权完美匹配的过程实际上是一个不断调整顶标的过程

我们初始化左部点顶标为对应出边最大值，右部点顶标为 0

我们先给出类似匈牙利算法的朴素思想，每次搜索假设我们左部点为 $u$，我们遍历其所有出边，如果右部点 $v$ 已被访问则跳过，否则检测该边是否在相等子图中，如果在子图中未被匹配或者递归可以更换匹配点的话（和匈牙利类似）, 我们返回增广成功的信息. 如果不在子图中，我们更新右部点的 $slack$ 值

然后取 $d$ 为所有未访问右部点的最小 $slack$ 值，访问过的左部点顶标减去 $d$, 右部点加上 $d$, 对每个点进行此循环直至成功为止，最终的所有顶标和即为最大权,$match$ 数组即为匹配情况，用 $dfs$ 实现复杂度 $O(n^4)$

```cpp
template <class T>
class KuhnMunkres
{
private:
    const T low = std::numeric_limits<T>::min() / 2;
    const T high = std::numeric_limits<T>::max() / 2;
    int n;                                  // 顶点数量
    std::vector<std::vector<T>> graph; // 邻接矩阵表示的带权二分图
    std::vector<T> A;                  // 左部顶点的顶标
    std::vector<T> B;                  // 右部顶点的顶标
    std::vector<int> match;            // 匹配结果，match[v] = u表示右部点v匹配左部点u
    std::vector<T> slack;              // 松弛量数组
    std::vector<int> vis;
    // 访问标记数组

    /**
     * @brief DFS寻找增广路
     * @param u 当前尝试匹配的左部顶点
     * @return 是否找到增广路
     */
    bool dfs(int u)
    {
        for (int v = 1; v <= n; ++v)
        {
            if (vis[v])
                continue; // 右部顶点已访问过

            if (A[u] + B[v] == graph[u][v])
            { // 满足相等子图条件
                vis[v] = true;
                if (!match[v] || dfs(match[v]))
                {                  // 如果v未匹配或可以找到增广路
                    match[v] = u; // 更新匹配
                    return true;
                }
            }
            else
            {
                // 更新松弛量
                slack[v] = std::min(slack[v], A[u] + B[v] - graph[u][v]);
            }
        }
        return false;
    }

public:
    /**
     * @brief 构造函数，初始化KM算法
     * @param n 顶点数量
     * @param edges 边的三元组(u, v, w)列表
     */
    KuhnMunkres(int n, const std::vector<std::tuple<int, int, int>> &edges)
        : n(n), graph(n + 1, std::vector<T>(n + 1, low)),
          A(n + 1, low), B(n + 1, 0),
          match(n + 1, 0), slack(n + 1, high), vis(n + 1, 0)
    {

        // 初始化邻接矩阵
        for (const auto &[u, v, w] : edges)
        {
            graph[u][v] = w;
```

```
        }

        // 初始化左部顶标为出边最大权值
        for (int u = 1; u <= n; ++u)
        {
            for (int v = 1; v <= n; ++v)
            {
                A[u] = std::max(A[u], graph[u][v]);
            }
        }
    }

    /**
     * @brief 计算最大权完美匹配
     * @return 匹配结果和总权重
     */
    std::pair<std::vector<int>, T> solve()
    {
        for (int u = 1; u <= n; ++u)
        { // 为每个左部顶点寻找匹配
            while (true)
            {
                // 每次尝试前重置访问标记和松弛量
                std::fill(vis.begin(), vis.end(), 0);
                std::fill(slack.begin(), slack.end(), high);

                if (dfs(u))
                    break; // 找到增广路则退出

                // 计算最小的松弛量d
                T d = high;
                for (int v = 1; v <= n; ++v)
                {
                    if (!vis[v])
                    {
                        d = std::min(d, slack[v]);
                    }
                }

                // 调整顶标
                for (int v = 1; v <= n; ++v)
                {
                    if (vis[v])
                    { // 在增广路中的右部顶点
                        B[v] += d;
                        A[match[v]] -= d;
                    }
                }
                A[u] -= d; // 调整当前左部顶点的顶标
            }
        }

        // 计算总权重
        T totalWeight = 0;
        for (int v = 1; v <= n; ++v)
        {
            if (match[v] > 0)
            {
                totalWeight += graph[match[v]][v];
            }
        }

        return {match, totalWeight};
    }
};
```

我们考虑继续优化，发现调整顶标后重新搜索十分浪费时间，我们考虑改 $dfs$ 为 $bfs$，具体的说就是利用了交错树遗留的信息优化了复杂度，时间复杂度为 $O(n^3)$，实际上 $bfs$ 的过程并不好理解，卡 $dfs$ 的题目也并不多

```cpp
template <class T>
class KuhnMunkresBFS
{
private:
    const T low = std::numeric_limits<T>::min() / 2;
    const T high = std::numeric_limits<T>::max() / 2;
    int n;                                  // 顶点数量
    std::vector<std::vector<T>> graph; // 邻接矩阵表示的带权二分图
    std::vector<T> A;                  // 左部顶点的顶标
    std::vector<T> B;                  // 右部顶点的顶标
    std::vector<int> match;            // 匹配结果，match[v] = u表示右部点v匹配左部点u
    std::vector<T> slack;              // 松弛量数组
    std::vector<int> vis;              // 访问标记数组
    std::vector<int> pre;              // 前驱数组，用于记录增广路径

    /**
     * @brief BFS寻找增广路
     * @param u 起始的左部顶点
     */
    void bfs(int u)
    {
        std::fill(vis.begin(), vis.end(), 0);        // 重置访问标记
        std::fill(slack.begin(), slack.end(), high); // 重置松弛量
        int x = u, y = 0; // x:当前左部点，y:当前右部点
        match[0] = u;     // 哨兵值，便于后续处理
        while (true)
        {
            vis[y] = true; // 标记当前右部点已访问

            // 寻找最小松弛量和对应的右部点
            T minSlack = high;
            int nextY = 0;
            for (int i = 1; i <= n; ++i)
            {
                if (vis[i])
                    continue; // 跳过已访问的右部点

                // 计算当前松弛量
                T currentSlack = A[x] + B[i] - graph[x][i];

                // 更新松弛量数组和前驱
                if (currentSlack < slack[i])
                {
                    slack[i] = currentSlack;
                    pre[i] = y; // 记录前驱
                }
                // 跟踪最小松弛量
                if (slack[i] < minSlack)
                {
                    minSlack = slack[i];
                    nextY = i;
                }
            }
            // 调整顶标
            A[u] -= minSlack; // 调整起始左部点顶标
            for (int i = 1; i <= n; ++i)
            {
                if (vis[i])
                {
                    // 对于已访问的右部点，调整其和匹配左部点的顶标
                    B[i] += minSlack;
                    A[match[i]] -= minSlack;
                }
                else
                {
                    // 对于未访问的右部点，减少松弛量
                    slack[i] -= minSlack;
                }
            }
            y = nextY; // 移动到最小松弛量对应的右部点

            if (match[y] == 0)
                break; // 找到未匹配点，可以增广

            x = match[y]; // 继续从匹配的左部点开始搜索
        }
```

```
        // 更新匹配关系
        while (y != 0)
        {
            match[y] = match[pre[y]]; // 沿着前驱链回溯更新匹配
            y = pre[y];
        }
    }
public:
    /**
     * @brief 构造函数，初始化KM算法
     * @param n 顶点数量
     * @param edges 边的三元组(u, v, w)列表
     */
    KuhnMunkresBFS(int n, const std::vector<std::tuple<int, int, int>> &edges)
        : n(n), graph(n + 1, std::vector<T>(n + 1, low)),
          A(n + 1, low), B(n + 1, 0),
          match(n + 1, 0), slack(n + 1), vis(n + 1), pre(n + 1)
    {
        // 初始化邻接矩阵
        for (const auto &[u, v, w] : edges)
        {
            graph[u][v] = w;
        }
        // 初始化左部顶标为出边最大权值
        for (int u = 1; u <= n; ++u)
        {
            for (int v = 1; v <= n; ++v)
            {
                if (graph[u][v] > A[u])
                {
                    A[u] = graph[u][v];
                }
            }
        }
    }
    /**
     * @brief 计算最大权完美匹配
     * @return 匹配结果和总权重
     */
    std::pair<std::vector<int>, T> solve()
    {
        for (int u = 1; u <= n; ++u)
        {
            bfs(u); // 为每个左部顶点寻找匹配
        }

        // 计算总权重
        T totalWeight = 0;
        for (int v = 1; v <= n; ++v)
        {
            if (match[v] > 0)
            {
                totalWeight += graph[match[v]][v];
            }
        }
        return {match, totalWeight};
    }
};
```

**0.24.2.6 二分图最大权匹配** 我们只需要将点数少的一边补足点，并且补满边权为 $0$ 或者负无穷大的边，即可跑 $KM$ 算法求最大权完美匹配

### 0.24.3 一般图最大匹配

在最大匹配的基础上，代码求出了匹配必需点（可能用于一般图上博弈）。

```
#include <bits/stdc++.h>
using namespace std;

struct Edmonds
{
    int n;
    vector<vector<int>> g;
    vector<int> match, base, p, q, inq, inb, ed;

    Edmonds(int n) : n(n), g(n) {}

    void addEdge(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int a, int b, vector<int> &mark)
    {
        fill(mark.begin(), mark.end(), 0);
        while (true)
        {
            a = base[a];
            mark[a] = 1;
            if (match[a] == -1)
                break;
            a = p[match[a]];
        }
        while (true)
        {
            b = base[b];
            if (mark[b])
                return b;
            if (match[b] == -1)
                break;
            b = p[match[b]];
        }
        return -1;
    }

    void markPath(int v, int b, int x)
    {
        while (base[v] != b)
        {
            inb[base[v]] = inb[base[match[v]]] = 1;
            p[v] = x;
            x = match[v];
            v = p[match[v]];
        }
    }

    int findPath(int root)
    {
        inq.assign(n, 0);
        p.assign(n, -1);
        iota(base.begin(), base.end(), 0);
        int qh = 0, qt = 0;
        q[qt++] = root;
        inq[root] = 1;
        while (qh < qt)
        {
            int v = q[qh++];
            for (int u : g[v])
                if (base[v] != base[u] && match[v] != u)
                {
                    if ((u == root) || (match[u] != -1 && p[match[u]] != -1))
                    {
                        int curbase = lca(v, u, ed);
                        fill(inb.begin(), inb.end(), 0);
                        markPath(v, curbase, u);
                        markPath(u, curbase, v);
                        for (int i = 0; i < n; i++)
                            if (inb[base[i]])
                            {
                                base[i] = curbase;
                                if (!inq[i])
```

```
                                q[qt++] = i, inq[i] = 1;
                            }
                        }
                        else if (p[u] == -1)
                        {
                            p[u] = v;
                            if (match[u] == -1)
                            {
                                while (u != -1)
                                {
                                    v = p[u];
                                    int w = match[v];
                                    match[v] = u;
                                    match[u] = v;
                                    u = w;
                                }
                                return 1;
                            }
                            else
                            {
                                q[qt++] = match[u];
                                inq[match[u]] = 1;
                            }
                        }
                    }
                }
                return 0;
            }

            int maxMatching()
            {
                match.assign(n, -1);
                base.resize(n);
                q.resize(n);
                inb.resize(n);
                ed.resize(n);
                int res = 0;
                for (int i = 0; i < n; i++)
                    if (match[i] == -1)
                        res += findPath(i);
                return res;
            }

            // 返回所有必需点
            vector<int> essentialVertices()
            {
                int M = maxMatching();

                // Dulmage-Mendelsohn 分解
                vector<int> vis(n, 0);
                queue<int> que;
                for (int i = 0; i < n; i++)
                    if (match[i] == -1)
                    {
                        que.push(i);
                        vis[i] = 1;
                    }

                while (!que.empty())
                {
                    int v = que.front();
                    que.pop();
                    for (int u : g[v])
                    {
                        int m = match[u];
                        if (!vis[u])
                        {
                            vis[u] = 1;
                            if (m != -1 && !vis[m])
                            {
                                vis[m] = 1;
                                que.push(m);
                            }
                        }
                    }
                }

                // 必需点：未被交替树覆盖的点
                vector<int> ans;
                for (int i = 0; i < n; i++)
```

```
        {
            if (!vis[i])
                ans.push_back(i);
        }
        return ans;
    }
};

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    Edmonds solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        solver.addEdge(u, v);
    }
    std::cout << solver.maxMatching() << '\n';
    auto t = solver.match;
    for (int i = 0; i < t.size(); ++i)
    {
        std::cout << t[i] + 1 << " \n"[i + 1 == t.size()];
    }
}
```

## 0.25 环计数问题

### 0.25.1 无向图简单环计数

复杂度 $O(n^2 * 2^n)$
由于无向图一个环顺时针和逆时针都会重复计数，所以 ans/=2
下面代码统计的为长度 >=3 的简单环个数

```
void solve(){
    int n,m;cin>>n>>m;
    vector<vector<int>> e(n);
    vector<vector<int>> vis(n,vector<int> (n));
    for(int i = 1;i<=m;i++){
        int u,v;cin>>u>>v;
        u--,v--;
        e[u].emplace_back(v);
        e[v].emplace_back(u);
        vis[u][v] = vis[v][u] = 1;
    }
    vector<vector<int>> dp(n,vector<int> (1LL<<n));
    for(int i = 0;i<n;i++){
        dp[i][1<<i] = 1;
    }
    for(int i = 1;i<(1<<n);i++){
        for(int k = __lg(i&-i);k<n;k++){
            if(i>>k&1) continue;
            for(int v:e[k]){
                if(i>>v&1){
                    dp[k][i|(1<<k)] += dp[v][i];
                }
            }
        }
    }
    int ans = 0;
    for(int i = 1;i<(1<<n);i++){
        if(__builtin_popcount(i)<=2) continue;
        for(int j = 0;j<n;j++){
            int k = __lg(i&-i);
            if(vis[j][k]) ans += dp[j][i];
        }
    }
    ans /= 2;
    cout<< ans<<"\n";
}
```

### 0.25.2 无向图三元环计数

首先给所有边定向。我们规定从度数小的点指向度数大的点，度数相同就从编号小的点指向编号大的点。那么此时此图是一张有向无环图。
然后我们枚举所有的点 $u$ 和 $u$ 的出点 $v$, 再枚举 $v$ 的出点 $w$，检查 $w$ 和 $u$ 是否联通即可。
可以证明复杂度为 $O(msqrt(m))$

```
class TriangleCounter
{
private:
    std::vector<std::vector<int>> adj;
    int n,m;
public:
    // 构造函数：接收顶点数和边数
    TriangleCounter(int vertices, int edges) : n(vertices), m(edges), adj(n + 1) {}
    void addEdge(int u, int v){// 添加边到图中
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    // 计算图中三角形的数量
    int countTriangles() const
    {
        int ans = 0;
        std::vector<int> vis(n + 1, 0);
        // 遍历每个顶点
        for (int u = 1; u <= n; ++u){
            // 第一阶段：标记满足条件的邻接点
            for (int v : adj[u]){
                if (shouldPointTo(u, v)){
                    vis[v] = u;
                }
            }
```

```
            // 第二阶段：查找三角形
            for (int v : adj[u]){
                if (shouldPointTo(u, v)){
                    for (int w : adj[v]){
                        if (shouldPointTo(v, w) && vis[w] == u)
                        {
                            ++ans;
                        }
                    }
                }
            }
        }
        return ans;
    }
private:
    // 判断边的方向：度数大的点指向度数小的点，或者度数相同则编号大的点指向编号小的点
    bool shouldPointTo(int u, int v) const
    {
        return adj[u].size() > adj[v].size() ||
               (adj[u].size() == adj[v].size() && u > v);
    }
};
```

### 0.25.3 无向图四元环计数

类似地，四元环 就是指四个点 $a.b.c.d$ 满足 $(a,b)(b,c)(c,d)(d,a)$ 均有边连接。
考虑先对点进行排序。度数小的排在前面，度数大的排在后面。
考虑枚举排在最后面的点 $a$，此时只需要对于每个比 $a$ 排名更前的点 $c$，都求出有多少个排名比 $a$ 前的点 $b$ 满足 $(a,b)(c,b)$ 有边。然后只需要这些中任取两个都能成为一个四元环。求 $b$ 的数量只需要遍历一遍 $b$ 和 $c$ 即可。
注意到我们枚举的复杂度本质上与枚举三元环等价，所以时间复杂度相同。
值得注意的是，$(a,b,c,d)$ 和 $(a,c,b,d)$ 可以是两个不同的四元环。
另外，度数相同的结点的排名将不相同，并且需要注意判断 $a!=c$
复杂度为 $O(msqrt(m))$

```
class SquareCounter
{
private:
    int n;                              // 顶点数
    std::vector<int> deg;               // 顶点度数
    std::vector<std::vector<int>> E;  // 原始无向图邻接表
    std::vector<std::vector<int>> E1; // 有向图邻接表（度数大→小，或编号大→小）
    std::vector<int> cnt;               // 临时计数器

public:
    // 构造函数：初始化图的大小
    explicit SquareCounter(int vertices)
        : n(vertices), deg(n + 1), E(n + 1), E1(n + 1), cnt(n + 1) {}

    // 添加边到图中
    void addEdge(int u, int v)
    {
        E[u].push_back(v);
        E[v].push_back(u);
        deg[u]++;
        deg[v]++;
    }

    // 计算图中三角形数量
    int count()
    {
        buildDirectedGraph();
        return calc();
    }

private:
    // 构建有向图E1：度数大的点指向度数小的点，度数相同则编号大的点指向编号小的点
    void buildDirectedGraph()
    {
        for (int u = 1; u <= n; ++u)
        {
            for (int v : E[u])
            {
                if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v))
                {
                    E1[u].push_back(v);
                }
            }
        }
    }

    // 处理并计算三角形数量
    int calc()
    {
        int ans = 0;
        for (int a = 1; a <= n; ++a)
        {
            // 遍历a的所有有向边邻居b
            for (int b : E1[a])
            {
                // 遍历b的所有邻居c
                for (int c : E[b])
                {
                    // 检查c是否满足条件：度数小于a或度数相同但编号小于a
                    if (deg[c] < deg[a] || (deg[c] == deg[a] && c < a))
                    {
                        ans += cnt[c]++;
                    }
                }
            }
            // 重置计数器
```

```
            for (int b : E1[a])
            {
                for (int c : E[b])
                {
                    cnt[c] = 0;
                }
            }
        }
        return ans;
    }
};
```

### 0.26 最大团与最大独立集

考虑到最大独立集是补图的最大团，因此我们只需要考虑怎么求最大团。

我们考虑求最大团并计数

$Solution1$ 折半搜索

我们考虑折半搜索，对前半部分，我们求出每个子集 $S$ 对应的最大团，以及对应的数量。这个操作可以使用高维前缀和完成，复杂度为 $O(2^{n/2} * n/2)$,但我们发现我们可以考虑线性递推，具体的说，我们每次拿出 $S$ 对应最低的位，考虑该位对应的点选不选即可。因此复杂度降为 $O(2^{n/2})$。对后半部分，我们只考虑判断每个子集是否为团，然后找出和前半部分的兼容掩码，更新答案即可，最终复杂度 $O(2^{n/2})$

```
using ll = long long;
struct Result
{
    int size;               // 最大规模（顶点数量）
    ll count;               // 解的总数量（所有满足最大规模的子集个数）
    std::vector<int> nodes; // 其中一个具体解（顶点编号集合）
};
// 图求解器类：封装最大团和最大独立集的求解逻辑
class GraphSolver
{
private:
    int n;                                  // 图的顶点总数（顶点编号从1开始）
    // 邻接矩阵：adj[u][v] = true 表示u和v之间有边
    std::vector<std::vector<bool>> adj;
    // 预处理的位计数表：popCount[mask] = mask二进制中1的个数（快速计算子集大小）
    std::vector<int> pop;
    // 预处理的低位1位置表：log2[lowbit] = 该低位1对应的幂次+1（快速定位顶点编号）
    std::vector<int> log2;
    // 最大位宽限制：确保log2表覆盖足够的掩码范围（1<<25可满足n<=50的场景）
    const int bit = 25;
public:
    // 构造函数：初始化顶点数量和邻接矩阵
    // 参数n：图的顶点总数
    GraphSolver(int n) : n(n), adj(n + 1, std::vector<bool>(n + 1, false))
    {
        // 1. 预处理位计数表popCount
        // 最大掩码长度为前半部分顶点数：(n+1)/2（折半搜索的前半段）
        int maxMask = 1 << ((n + 1) / 2);
        pop.resize(maxMask, 0);
        // 递推计算每个掩码的1的个数：mask的1的个数 = (mask去掉最低位1后的掩码)的1的个数 + 1
        for (int i = 1; i < maxMask; ++i)
        {
            pop[i] = pop[i ^ (i & -i)] + 1;
        }
        // 2. 预处理log2表（定位最低位1对应的顶点）
        log2.resize(1 << bit, 0);
        // 对于2^i的掩码，其最低位1在第i位，对应顶点编号i+1（因顶点从1开始）
        for (int i = 0; i < bit; ++i)
        {
            log2[1 << i] = i + 1;
        }
    }
    void addEdge(int u, int v)
    {
        adj[u][v] = true; // u到v有边
        adj[v][u] = true; // 无向图，v到u也有边
    }
    Result solve0() // 最大团
    {
        return solve(false);
    }
    Result solve1()
    {
        return solve(true);
    }
private:
    // 核心求解函数：统一处理最大团（原图）和最大独立集（补图）
    Result solve(bool ok)
    {
        std::vector<std::vector<bool>> cur; // 当前使用的邻接矩阵（原图或补图）
        if (ok)
        {
            // 构建补图：补图中u和v有边 <-> 原图中u和v无边（自身环设为true，不影响团的判断）
            cur.resize(n + 1, std::vector<bool>(n + 1, false));
            for (int i = 1; i <= n; ++i)
            {
                for (int j = 1; j <= n; ++j)
                {
                    if (i == j){
```

```
                cur[i][j] = true; // 自身环不影响团（团要求任意两点有边，自身无需考虑）
            }
            else{
                cur[i][j] = !adj[i][j]; // 补图边 = 原图边取反
            }
        }
    }
}
else
{
    cur = adj; // 求解最大团，直接使用原图邻接矩阵
}

// -------------------------- 步骤2：预处理顶点邻接掩码 --------------------------
int mid = n / 2;                // 折半搜索：将顶点分为前半段（1~mid）和后半段（mid+1~n）
std::vector<ll> mask(n + 1, 0); // mask[i]：顶点i的邻接顶点掩码（二进制位j-1=1表示i与j有边）
for (int i = 1; i <= n; ++i)
{
    for (int j = 1; j <= n; ++j)
    {
        if (cur[i][j])
        {
            // 将j-1位设为1（因顶点j对应二进制位j-1，从0开始计数）
            mask[i] |= 1LL << (j - 1);
        }
    }
}

// -------------------------- 步骤3：处理前半段顶点（动态规划预处理） --------------------------
int x = 1 << mid; // 前半段顶点的所有子集数（2^mid个）
// f[s]：前半段子集s对应的"最优团信息"
// first：该子集能扩展出的最大团的掩码，second：该最大团的计数
std::vector<std::pair<int, ll>> f(x);
f[0] = {0, 1}; // 空集的初始状态：最大团为空（掩码0），计数1
// 枚举前半段所有非空子集
for (int s = 1; s < x; ++s)
{
    int lowbit = s & -s;   // 取子集s的最低位1（对应一个顶点）
    int prev = s ^ lowbit; // 去掉最低位1后的子集（前一个状态）
    int v = log2[lowbit];  // 最低位1对应的顶点编号（从1开始）
    // 两种选择：
    // op1：不选顶点v，直接沿用prev子集的最优团信息
    auto op1 = f[prev];
    // op2：选顶点v，此时需取prev与v邻接的子集（确保v与团中所有顶点有边），再加入v
    auto op2 = f[prev & mask[v]];
    op2.first |= lowbit; // 将v对应的位加入团掩码
    // 选择更优的方案（优先按团大小，再按计数累加）
    if (pop[op2.first] > pop[op1.first])
    {
        // op2的团更大，更新为op2
        f[s] = op2;
    }
    else if (pop[op2.first] == pop[op1.first])
    {
        // 团大小相同，计数累加
        f[s].first = op1.first;
        f[s].second = op1.second + op2.second;
    }
    else
    {
        // op1的团更大，更新为op1
        f[s] = op1;
    }
}

// -------------------------- 步骤4：处理后半段顶点（筛选有效团） --------------------------
int y = 1 << (n - mid);             // 后半段顶点的所有子集数（2^(n-mid)个）
std::vector<bool> valid(y, false); // valid[s]：标记后半段子集s是否为团
valid[0] = true;                    // 空集是团
// 枚举后半段所有非空子集
for (int s = 1; s < y; ++s)
{
    int lowbit = s & -s;        // 取子集s的最低位1
    int prev = s ^ lowbit;      // 去掉最低位1后的子集
    int v = log2[lowbit] + mid; // 转换为原顶点编号（后半段顶点从mid+1开始）
    // 判断子集s是否为团：
    // 1. 前一个状态prev是团（valid[prev]为true）
    // 2. 新加入的顶点v与s中所有其他顶点有边（通过掩码判断：s去掉v后的部分都在v的邻接集中）
```

```
            valid[s] = valid[prev] && ((prev & ~(mask[v] >> mid)) == 0);
        }

        // -------------------------- 步骤5：合并前后半段结果，找全局最优解 --------------------------
        int maxSize = 0;           // 全局最大团/独立集的规模
        ll count = 0;              // 全局最大解的总数量
        ll bestMask = 0;           // 全局最优解的顶点掩码（二进制位i=1表示顶点i+1在解中）
        std::vector<int> to(y, 0); // to[b]：后半段子集b对应的前半段兼容最大团的掩码

        // 枚举所有有效的后半段团
        for (int b = 0; b < y; ++b)
        {
            if (!valid[b])
                continue; // 跳过非团的子集

            // 计算前半段中与后半段子集b兼容的最大团掩码t：
            // 兼容指：t中的所有顶点与b中的所有顶点都有边
            int t;
            if (b == 0)
            {
                // 后半段是空集，前半段可兼容的最大团是前半段所有顶点（掩码(1<<mid)-1）
                t = (1 << mid) - 1;
            }
            else
            {
                int lowbit = b & -b;         // 取后半段子集b的最低位1
                int prev = b ^ lowbit;       // 去掉最低位1后的子集
                int v = log2[lowbit] + mid; // 转换为原顶点编号
                // 前半段兼容掩码 = 前一个状态的兼容掩码（交）v的邻接掩码（确保与新加入的v兼容）
                t = to[prev] & mask[v];
            }
            to[b] = t; // 记录当前后半段子集b对应的前半段兼容掩码
            // 计算当前合并后的团规模和计数
            int curSize = pop[f[t].first] + pop[b]; // 前半段最大团大小 + 后半段团大小
            ll curCount = f[t].second;              // 前半段最大团的计数（后半段团只有1种）
            // 更新全局最优解
            if (curSize > maxSize)
            {
                // 找到更大的解，更新规模、计数和掩码
                maxSize = curSize;
                count = curCount;
                // 合并前后半段掩码：前半段掩码直接用，后半段掩码左移mid位（对应原顶点编号）
                bestMask = f[t].first | (ll(b) << mid);
            }
            else if (curSize == maxSize)
            {
                // 与当前最大规模相同，累加计数
                count += curCount;
            }
        }
        // -------------------------- 步骤6：从最优掩码中提取顶点编号 --------------------------
        std::vector<int> nodes;
        for (int i = 1; i <= n; ++i)
        {
            // 检查第i个顶点是否在最优掩码中（对应二进制位i-1）
            if (bestMask & (1ULL << (i - 1)))
            {
                nodes.push_back(i);
            }
        }
        // 返回求解结果
        return {maxSize, count, nodes};
    }
};
```

$Solution2$ 启发式搜索

我们考虑每次找到最大的度数 $max$, 如果 $max \leq 2$，说明是环或者链，可以通过预处理好的 $f$ 和 $g$ 解决，否则我们选择度数最大的点递归搜索。

复杂度可以证明，上限为 $O(1.38^n)$，实际运行速度极快

```cpp
#include <bits/stdc++.h>

using u64 = unsigned long long;
const int N = 55; // 最大顶点数量
int n, m;         // 顶点数和边数
int f[N], g[N];   // 存储链和环的方案数
u64 U;            // 表示所有顶点的集合（位掩码）
u64 e[N];         // 邻接表（位掩码形式）

/**
 * 深度优先搜索寻找最大独立集
 * @param S 当前待处理的顶点集合（位掩码）
 * @return  pair<u64, int>  最大独立集及其方案数
 */
std::pair<u64, int> dfs(u64 S)
{
    if (!S)
        return {0, 1}; // 空集处理

    // 选择当前集合中度数最大的顶点作为分支点
    int w = 0, d = -1;
    for (u64 i = S; i; i &= (i - 1))
    {
        int x = __builtin_ctzll(i);
        int t = __builtin_popcountll(e[x] & S); // 计算当前顶点在集合S中的度数
        if (t > d)
        {
            d = t;
            w = x;
        }
    }

    // 处理度数<=2的特殊情况（链或环）
    if (d <= 2)
    {
        u64 v = S, x = 0, s1, s2;
        int y = 1;
        bool O;

        // 内部DFS用于探索连通分量并进行二分图着色
        auto dfs = [&](auto &&sf, int x, bool o) -> void
        {
            v ^= (1ull << x);                           // 标记顶点x为已处理
            (o ? s2 : s1) |= 1ull << x;                 // 按颜色分组
            O &= (__builtin_popcountll(e[x] & S) == 2); // 检查是否为环
            // 递归访问所有未处理的邻居
            while (v & e[x])
            {
                sf(sf, __builtin_ctzll(v & e[x]), o ^ 1);
            }
        };

        // 处理所有连通分量
        while (v)
        {
            s1 = s2 = 0;
            O = 1;                              // 初始假设为环
            dfs(dfs, __builtin_ctzll(v), 0); // 从最低位顶点开始处理

            int a = __builtin_popcountll(s1);
            int b = __builtin_popcountll(s2);
            int num = a + b;

            // 选择最大独立集并累计算法数
            if (O && (num & 1))
            { // 奇环
                x |= (a < b ? s1 : s2);
            }
            else
            { // 偶环或链
                x |= (a > b ? s1 : s2);
            }
            y *= (O ? g : f)[num];
```

```
        }
        return {x, y};
    }

    // 常规情况：分支递归
    u64 W = 1ull << w;
    std::pair<u64, int> nw = dfs(S ^ W); // 不选顶点w

    // 选择顶点w（需排除其邻居）
    auto [x, y] = dfs(S & (~(e[w] | W)));
    x |= W;

    // 比较两种情况，选择最优解
    if (__builtin_popcountll(x) > __builtin_popcountll(nw.first))
    {
        nw = {x, y};
    }
    else if (__builtin_popcountll(x) == __builtin_popcountll(nw.first))
    {
        nw.second += y;
    }

    return nw;
}
// 预处理f和g数组（链和环的方案数）
void preprocess()
{
    // f[i]：i个顶点的链的最大独立集方案数
    // g[i]：i个顶点的环的最大独立集方案数
    for (int i = 1; i <= n; ++i)
    {
        f[i] = (i & 1) ? 1 : (i / 2 + 1); // 链的方案数公式
        g[i] = (i & 1) ? i : 2;           // 环的方案数公式
    }
}
void exchange() // 翻转图
{
    for (int i = 0; i < n; ++i)
    {
        // 补图的邻接集 = 全集排除自身和原图邻接集
        e[i] = ((~e[i]) & U) ^ (1ull << i);
    }
}

int main()
{
    std::ios::sync_with_stdio(0);
    std::cin.tie(0);

    std::cin >> n >> m;
    U = (1ull << n) - 1;
    while (m--)
    {
        int u, v;
        std::cin >> u >> v;
        --u;
        --v;
        e[u] |= (1ull << v);
        e[v] |= (1ull << u);
    }
    preprocess();
    exchange();
    auto [val, cnt] = dfs(U);
    std::cout << __builtin_popcountll(val) << ' ' << cnt << '\n';
    while (val)
    {
        std::cout << __builtin_ctzll(val) + 1 << ' ';
        val ^= (val & (-val));
    }
    std::cout << '\n';
    exchange();
    auto [x, y] = dfs(U);
    val = x, cnt = y;
    std::cout << __builtin_popcountll(val) << ' ' << cnt << '\n';
    while (val)
    {
        std::cout << __builtin_ctzll(val) + 1 << ' ';
        val ^= (val & (-val));
    }
}
```